Table of Contents

GigaDigDefTopic

< >

About the GigaDig

About the GigaDig

The GigaDig for UltraFLEX test systems is a differential, undersampling digitizer with eight channels and an 8 GHz bandwidth. Typical uses for the GigaDig include testing high speed datacommunication, LAN, and storage products.

This section includes the following topics:

- Features
- Safety Information
- Key Terms
- GigaDig Software Licensing Requirements
- GigaDig Calibration

GigaDig information includes the following sections:

- GigaDig Theory of Operation
- GigaDig Programming
- GigaDig Debug Displays
- GigaDig Examples

Related Topics

- GigaDig Specifications
- GigaDig (VBT syntax)
- GigaDig DIB Design Guidelines

< >

gigadig_about.1.2

< >

About the GigaDig : Features

Features
GigaDig features include:

- Multiple input ranges:
  - 64mVpkd (volts peak differential)
  - 128mVpkd
  - 256mVpkd
  - 512mVpkd
  - 1.024Vpkd
  - 2.048Vpkd
  - 3.072Vpkd
- Sampling rate programmable from 10MHz to 20MHz
- Variable resolution of samples, up to 14 bits; maximum resolution is the number of ADC bits or the memory size divided by the number of samples
- Sample size from 1 to 8 Megasamples per channel
- Triggered operation, allowing you to start the GigaDig from:
  - A test program using VBT language
  - A digital pattern using the TRIG microcode

- Selective capture in block mode:
  - Programmable sample size allows you to specify the number of samples to capture in each block
  - Programmable skip count allows you to specify the number of samples to skip between blocks
- Parallel capture: Each of the eight GigaDig instruments on each board can:
  - Trigger independently
  - Capture simultaneously
- Support for up to four boards per test system for a total of 32 channels
- DSP support
- Data movement from capture memory by:
  - Program Initiated Move (PIM): Move data to host computer under test program control when the capture is complete
  - Instrument Initiated Move (IIM): Move data to the DSP computer as soon as data is available. The data can be moved while the GigaDig continues to capture and regardless of what the other GigaDig instruments on the board or other boards are doing.
- Out-of-order undersampling with the ability to reorder samples automatically, before DSP processing, when you set the undersampling ratio and enable reordering
- Data Analysis that allows you to view captured data in WaveScope, showing the captured data with the appropriate voltage scale factor
- Alarms, programmable
  - Overrange alarms: one alarm per capture, regardless of the number of samples that exceed the range limit
  - Masking allowed: you can enable or disable the alarms.
- Reporting of errors for invalid programming values and combinations of values that can damage hardware; IG-XL prevents programming these values
- Automatic calibration by test system

- Pin Parametric Measurement Units on each channel (16 per board)

- Condition bit programming support

- Parameter sets (PSets) support

For GigaDig hardware specifications, see GigaDig Specifications.

< >

2015 Teradyne, Inc. All rights reserved.

gigadig_about.1.3

< >

About the GigaDig : Safety Information

Safety Information

When operating a GigaDig, always follow general UltraFLEX test system safety guidelines. See Test System Safety.

< >

gigadig_about.1.4

< >

About the GigaDig : Key Terms

Key Terms

actual sample rate	The rate at which the GigaDig physically captures samples from the device. Limited to 10MHz to 20MHz.
CMEM	Capture Memory. The memory in which the GigaDIg stores data during a capture. After the capture completes, the instrument moves the data out of CMEM for processing.
effective sample rate	The rate, after reconstructing an undersampled signal, at which the GigaDig appears to capture samples from the device.
HSD	High Speed Digital
IIM	Instrument Initiated Move. A protocol by which the GigaDig automatically moves captured data to the DSP Processor when the capture completes without requiring additional programming.
PPMU	Pin Parametric Measurement Unit
undersampling ratio	The ratio of the effective sample rate to the actual hardware sample rate. Used to rearrange samples from a capture using out-of-order undersampling.

< >

gigadig_about.1.5

< >

About the GigaDig : GigaDig Software Licensing Requirements

GigaDig Software Licensing Requirements

Use of GigaDig does not require a separate license.

For more information on IG-XL licensing, see IG-XL Licensing in IG-XL Administration help.

< >

gigadig_about.1.6

< >

About the GigaDig : GigaDig Calibration

GigaDig Calibration

To provide reliable readings, the GigaDig must be calibrated. IG-XL manages autocal for the instrument and calibrates the following aspects of the GigaDig:

- Gain (amplitude)
- Offset (removal of DC offset in the analog signal to maximize the dynamic range of the capture)
- VGA (ensures that the capture is within expected range)
- PPMU function

IG-XL automatically calibrates the GigaDig when necessary. It does this at least once every three days (72 hours), or more frequently if the test head temperature varies by more than 3 degrees Centigrade.

At run time, IG-XL performs autocal calibration to correct offset values and to calibrate the PPMUs.

For more information, see About Calibration.

< >

gigadig_apps.5.01

< >

GigaDig Examples

GigaDig Examples

This section contains the following kinds of program examples for the GigaDig:

- IG-XL workbook examples show all IG-XL data, VBT code, and patterns necessary for a complete running program.
- VBT function examples show one or more VBT functions that perform a programmed action when called as specified.

The following topics are available:

- Capture a Signal Using Pattern Control and PSets
- Capture an HSD Clock Signal Using Undersampling
- VBT Function Examples

Related Topics

- GigaDig (VBT Syntax Reference)
- GigaDig Programming

< >

gigadig_apps.5.02

< >

GigaDig Examples : Capture a Signal Using Pattern Control and PSets

Capture a Signal Using Pattern Control and PSets

This example sources a waveform from an AWG6G and captures the same waveform using the GigaDig. The test program uses a pattern to synchronize and trigger both instruments.

The GigaDig board that performs the capture is in slot 0. The AWG6G board that sources the waveform is in slot 1. The HSD1000 board that controls the pattern is in slot 16.

This workbook example includes the following program components:

- DUT Connections
- DataTool Sheets
- HSD Pattern with TSet and Pins
- VBT Code
- DSP Code

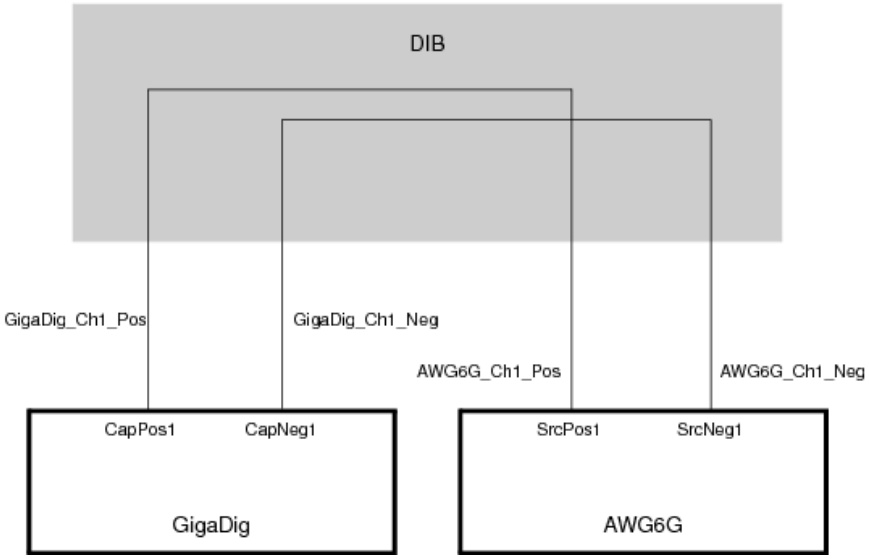
< >

gigadig_apps.5.03

< >

GigaDig Examples : Capture a Signal Using Pattern Control and PSets : DUT Connections

DUT Connections



GigaDig and AWG6G DIB Loop Connections

< >

gigadig_apps.5.04

< >

GigaDig Examples : Capture a Signal Using Pattern Control and PSets : DataTool Sheets

DataTool Sheets

This workbook example uses the following DataTool sheets:

- Pin Map Sheet
- Channel Map Sheet
- Time Sets (Basic) Sheet
- Pin Levels Sheet
- Test Instances Sheet
- Flow Table Sheet
- Global Specs
- Mixed-Signal Timing Sheet
- Wave Definitions Sheet

Pin Map Sheet

Pin Map

Group Name	Pin Name	Type	Comment
	Gigadig_ch1_pc	Analog	gigadig channel 1 pos
	Gigadig_ch1_ne	Analog	gigadig channel 1 neg
	AWG_ch1_pos	Analog	awg channel 1 pos
	AWG_ch1_neg	Analog	awg channel 1 neg
	hsd_ch0	I/O	digital channel 1
	hsd_ch1	I/O	digital channel 2
Gigadig_Pins	Gigadig_ch1_pc	Analog	gigadig ch 1 pin group
Gigadig_Pins	Gigadig_ch1_neg		
AWG_Pins	AWG_ch1_pos	Analog	awg ch 1 pin group
AWG_Pins	AWG_ch1_neg		
HSD_Pins	hsd_ch0	I/O	digital ch 1 and 2 pin group
HSD_Pins	hsd_ch1		

Channel Map Sheet

Channel Map

DIB ID: **View Mode:** Signal

Device Under Test		Tester Channel		
Pin Name	Package Pin	Type	Site 0	Comme
Gigadig_ch1_pos		GigaDigPos	0.cappos1	
Gigadig_ch1_neg		GigaDigNeg	0.capneg1	
AWG_ch1_neg		AWGPos	1.srcpos1	
AWG_ch1_pos		AWGNeg	1.srcneg1	
hsd_ch0		I/O	16.ch0	
hsd_ch1		I/O	16.ch1	

Time Sets (Basic) Sheet

Time Sets (Basic)

Timing Mode: Single **Master Timeset Name:**
Time Domain: **Strobe Ref Setup Name:**

	Cycle	Pin/Group			Data		Drive				Compare				Edge Resolution
Time Set	Period	Name	Clock Period	Setup	Src	Fmt	On	Data	Return	Off	Mode	Open	Close	Ref Offset	Mode
tset_1	50.E-09	HSD_Pins		i/o	PAT	NR		0		0	Off				Auto

Pin Levels Sheet

Pin Levels

Pin/Group	Seq.	Parameter	Value
HSD_Pins		Vil	0
HSD_Pins		Vih	1
HSD_Pins		Vol	0
HSD_Pins		Voh	1
HSD_Pins		VCmpMain	0
HSD_Pins		Vt	0
HSD_Pins		Vcl	-1
HSD_Pins		Vch	6
HSD_Pins		DriverMode	Largeswing-HiZ

Test Instances Sheet

Test Instances

	Test Procedure			DC Specs		AC Specs		Sheet Parameters				
Test Name	Type	Name	Called As	Category	Selector	Category	Selector	Time Sets	Edge Sets	Pin Levels	Mixed Signal Timing	Overlay
Capture_Analog_Signal	VBT	AnalogLoopback						TSB		Levels	CaptureWave	

Flow Table Sheet

Flow Table

Flow Domain:

		Gate			Command				Limits		Datalog Display Results			Bin Number		Sort Number		
Label	Enable	Job	Part	Env	Opcode	Parameter	TName	TNum	LoLim	HiLim	Scale	Units	Format	Pass	Fail	Pass	Fail	Result
					Test	Capture_Analog_Signal		101										
					Use-Limit	Capture_Analog_Signal	THD		-100	-30								
					Use-Limit	Capture_Analog_Signal	SNR		0	100								

Global Specs

Global Specs

Symbol	Job	Value	Comment
Vcl_default		– 1.E+00	Detector clamp voltage low
Vch_default		6.E+00	Detector clamp voltage high

Mixed-Signal Timing Sheet

Mixed Signal Timing

		Resource		Clocking					Instrument	Waveform	
Set Name	Subset	Type	ID	Fs	N	Fr	M	USR	Data	Definition	Filter
CaptureWave	Subset1	GigaDig		16665039.22	2048	250000000	3	10241	ReorderedSamples=1		
CaptureWave	Subset1	AWG		6000000000	192	250000000	8	1		aSine	

Wave Definitions Sheet

Wave Definitions

WaveDefName	WaveDefType	WaveDef Component	Repeat Count	Relative Period	Relative Amplitude	Relative Offset	Primitive Parameters
aSine	Primitive	Sine	1	1.E+00	1.E+00	0	phase=90

< >

2015 Teradyne, Inc. All rights reserved.

gigadig_apps.5.05

< >

GigaDig Examples : Capture a Signal Using Pattern Control and PSets : HSD Pattern with TSet and Pins

HSD Pattern with TSet and Pins

The following pattern starts the AWG6G and sources the waveform, then starts the GigaDig and captures the waveform.

TSet	Command	Label	Vector	AWG_ch1_pos (AWG)	Gigadig_ch1_pos (GigaDig)
TSet_1		start_lab…	0		
TSet_1			1	PSet AWG_PSet	PSet Gigadig_PSet
TSet_1	repeat 60000		2		
TSet_1	repeat 60000		3		
TSet_1	repeat 60000		4		
TSet_1	repeat 60000		5		
TSet_1			6	Resync	Resync
TSet_1	repeat 60000		7		
TSet_1	repeat 60000		8		
TSet_1	repeat 60000		9		
TSet_1	repeat 60000		10		
TSet_1			11	Start	
TSet_1			12		Enable_Inst_Cond
TSet_1	branch_expr = (!inst_co…		13		
TSet_1			14		Trig
TSet_1			15		
TSet_1		capture_l…	16		
TSet_1	if (branch_expr) jump c…		17		
TSet_1			18		Disable_Inst_Cond
TSet_1			19	Stop	
TSet_1	halt		20		

< >

gigadig_apps.5.06

< >

GigaDig Examples : Capture a Signal Using Pattern Control and PSets : VBT Code

VBT Code

The following VBT function creates source and capture signals, configures PSets for the AWG6G and GigaDig, sources the waveform from the AWG6G, captures the waveform using the GigaDig, calls a DSP procedure to analyze the capture, and tests it against limits from the Flow Table.

```
Public Function AnalogLoopback() As Long
Dim pat_name As String
pat_name = ".\AWG_Gigadig_Loopback_Analog.pat"
Dim THD As New SiteDouble
Dim SNR As New SiteDouble
' First unload to clear memory
TheHdw.AWG("AWG_Pins").Signals.Unload
TheHdw.AWG("AWG_Pins").Signals.Add ("AWG_Signal")
With TheHdw.AWG("AWG_Pins").Signals("AWG_Signal")
    .WaveDefinitionName = "aSine"
    .Amplitude = 0.25
    .CommonMode = 0#
    .Filter.LowPass.Bypass = False
    .ConnectionType = tlAWGConnectionTypeSingleEnded
    .LoadImpedance = 50
    .Calibration.LevelFrequency = 500000000
End With
' Connect positive channel of the board
TheHdw.AWG("AWG_Pins").Connect tlAWGFromPOS, tlAWGToDUT
' Create and configure PSet for AWG6G waveform
TheHdw.AWG("AWG_Pins").PSets.Add ("AWG_PSet")
With TheHdw.AWG("AWG_Pins").PSets("AWG_PSet")
    .Amplitude = 0.25
    .Calibration.LevelFrequency = 500000000
    .SampleRate = 6000000000#    ' from MST
End With
' Create and configure capture signal
TheHdw.GigaDig("GigaDig_Pins").Signals.Add ("GigaDig_Signal")
With TheHdw.GigaDig("GigaDig_Pins").Signals("GigaDig_Signal")
    .VoltageRange = 1.024
    .BlockCount = 1
```

```
    .BlockSpacing = 0
    .ExtraSamples = 0
End With
' Create and configure PSet for GigaDig capture
TheHdw.GigaDig("GigaDig_Pins").PSets.Add ("GigaDig_PSet")
With TheHdw.GigaDig("GigaDig_Pins").PSets("GigaDig_PSet")
    .BlockCount = 1
    .BlockSpacing = 0
    .ExtraSamples = 0
    .ReorderedSamples = True
    .VoltageRange = 1.024
    .SampleRate = 16665039.22
    .SampleSize = 2048
    .UnderSamplingRatio = 10241
End With
' Apply the AWG and GigaDigHD signals and Mixed Signal Timing solutions
TheHdw.AWG("AWG_Pins").Signals("AWG_Signal").ApplyMixedSignalTiming
TheHdw.AWG("AWG_Pins").Signals("AWG_Signal").LoadSettings
TheHdw.AWG("AWG_Pins").Signals.DefaultSignal = "AWG_Signal"
TheHdw.GigaDig("GigaDig_Pins").Signals("GigaDig_Signal").ApplyMixedSignalTiming
TheHdw.GigaDig("GigaDig_Pins").Signals("GigaDig_Signal").LoadSettings
TheHdw.GigaDig("GigaDig_Pins").Signals.DefaultSignal = "GigaDig_Signal"
' Apply levels and timing and load pattern
TheHdw.Digital.ApplyLevelsTiming True, True, True, tlPowered
TheHdw.Patterns(pat_name).Load
TheHdw.Patterns(pat_name).Start
Dim timer As Boolean
timer = TheHdw.GigaDig("GigaDig_Pins").Signals.WaitForCaptureDone
If timer = False Then
    MsgBox "Capture Timed Out"
    Stop
    Exit Function
Else
    If TheHdw.Digital.Patgen.IsRunning Then
        TheHdw.Digital.Patgen.Halt
    End If
End If
' Bind the captured data of GigaDigHD ch 1 pos to the local DSPwave
Dim CaptureData_1p As New DSPWave
CaptureData_1p = TheHdw.GigaDig("GigaDig_ch1_pos").Signals("GigaDig_Signal").DSPWave
' Call the embedded DSP function
rundsp.LoopbackAnalysis CaptureData_1p, THD, SNR
' Test the results against the limits specified in the flow table
TheExec.Flow.TestLimit resultval:=THD, unit:=Db, forceresults:=tlForceFlow
TheExec.Flow.TestLimit resultval:=SNR, unit:=Db, forceresults:=tlForceFlow
TheHdw.AWG("AWG_Pins").Disconnect tlAWGFromInst, tlAWGToDUT
    On Error GoTo errHandler
```

```
    Exit Function
errHandler:
    If AbortTest Then Exit Function Else Resume Next
End Function
```

< >

2015 Teradyne, Inc. All rights reserved.

gigadig_apps.5.07

< >

GigaDig Examples : Capture a Signal Using Pattern Control and PSets : DSP Code

DSP Code

```
Public Function LoopbackAnalysis(ByVal InputWave As DSPWave, _
                    ByRef retTHD As Double, _
                    ByRef retSNR As Double) As Long
Dim SpectrumWave As New DSPWave
   Set SpectrumWave = InputWave.Spectrum
   retTHD = SpectrumWave.CalcTHD
   retSNR = SpectrumWave.CalcSNR
End Function
```

< >

gigadig_apps.5.08

< >

GigaDig Examples : Capture an HSD Clock Signal Using Undersampling

Capture an HSD Clock Signal Using Undersampling

This example sources a clock signal from an HSD channel and captures the clock as a waveform using the GigaDig. The test program starts the pattern and triggers the capture using VBT.

This test program performs out-of-order undersampling and uses a function to calculate the sampling parameters for the GigaDig.

The GigaDig board that performs the capture is in slot 0. The HSD1000 board that controls the pattern is in slot 16.

This workbook example includes the following program components:

- DUT Connections
- DataTool Sheets
- HSD Pattern with TSet and Pins
- VBT Code
- DSP Code

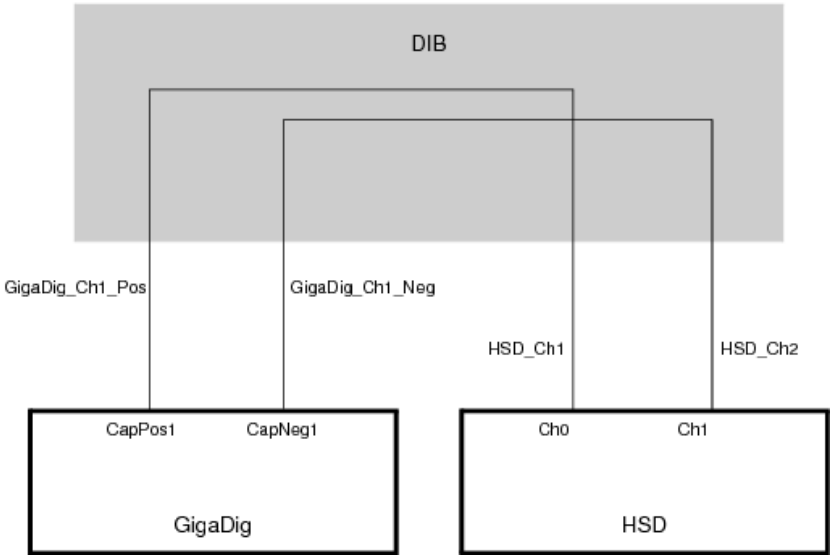
< >

gigadig_apps.5.09

< >

GigaDig Examples : Capture an HSD Clock Signal Using Undersampling : DUT Connections

DUT Connections



GigaDig and HSD DIB Loop Connections

< >

gigadig_apps.5.10

< >

GigaDig Examples : Capture an HSD Clock Signal Using Undersampling : DataTool Sheets

DataTool Sheets

This workbook example uses the following DataTool sheets:

- Pin Map Sheet
- Channel Map Sheet
- Time Sets (Basic) Sheet
- Pin Levels Sheet
- Test Instances Sheet
- Flow Table Sheet
- Global Specs
- Mixed-Signal Timing Sheet
- Wave Definitions Sheet

Pin Map Sheet

Pin Map

Group Name	Pin Name	Type	Comment
	Gigadig_ch1_pc	Analog	gigadig channel 1 pos
	Gigadig_ch1_ne	Analog	gigadig channel 1 neg
	hsd_ch1	I/O	digital channel 1
	hsd_ch2	I/O	digital channel 2
Gigadig_Pins	Gigadig_ch1_pc	Analog	gigadig ch 1 pin group
Gigadig_Pins	Gigadig_ch1_neg		
HSD_grp	hsd_ch1	I/O	
HSD_Pins	hsd_ch1	I/O	digital ch 1 and 2 pin group
HSD_Pins	hsd_ch2		

Channel Map Sheet

Channel Map

DIB ID: **View Mode:** Signal

Device Under Test		Tester Channel		
Pin Name	Package Pin	Type	Site 0	Comm
Gigadig_ch1_pos		GigaDigPos	0.cappos1	
Gigadig_ch1_neg		GigaDigNeg	0.capneg1	
hsd_ch1		I/O	16.ch0	
hsd_ch2		I/O	16.ch1	

Time Sets (Basic) Sheet

Time Sets (Basic)

Timing Mode: Single **Master Timeset Name:**
Time Domain: **Strobe Ref Setup Name:**

	Cycle	Pin/Group			Data		Drive				Compare				Edge Resolution
Time Set	Period	Name	Clock Period	Setup	Src	Fmt	On	Data	Return	Off	Mode	Open	Close	Ref Offset	Mode
tset_1	#####	HSD_Pins		i/o	PAT	NR		0		0	Off				Auto

Pin Levels Sheet

Pin Levels

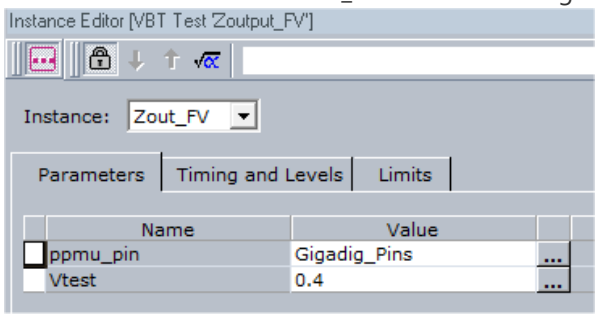
Pin/Group	Seq.	Parameter	Value
HSD_Pins		Vil	0
HSD_Pins		Vih	1
HSD_Pins		Vol	0
HSD_Pins		Voh	1
HSD_Pins		VCmpMain	0
HSD_Pins		Vt	0
HSD_Pins		Vcl	-1
HSD_Pins		Vch	6
HSD_Pins		DriverMode	Largeswing-HiZ

Test Instances Sheet

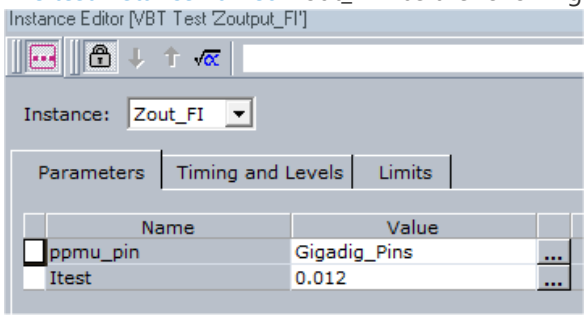
Test Instances

	Test Procedure			DC Specs		AC Specs		Sheet Parameters				
Test Name	Type	Name	Called As	Category	Selector	Category	Selector	Time Sets	Edge Sets	Pin Levels	Mixed Signal Timing	Overlay
Capture_Digital_Signal	VBT	DigitalLoopback						TSB		Levels		
Zout_FI	VBT	Zoutput_FI										
Zout_FV	VBT	Zoutput_FV										

The test instance named Zout_FI has the following test-specific parameters and values as shown in the Instance Editor:



The test instance named Zout_FV has the following test-specific parameters and values as shown in the Instance Editor:



Flow Table Sheet

Flow Table

Flow Domain:

		Gate			Command				Limits		Datalog Display			Bin Number		Sort Number		
Label	Enable	Job	Part	Env	Opcode	Parameter	TName	TNum	LoLim	HiLim	Scale	Units	Format	Pass	Fail	Pass	Fail	Result
					set-error-bin										999		999	
					Test	Capture_Digital_Signal		1	1									
					Use-Limit	Capture_Digital_Signal	Rise Time		0	400					2		2	Fail
					Use-Limit	Capture_Digital_Signal	Fall Time		0	200					3		3	Fail
					Test	Zout_FI		10	10									
					Use-Limit	Zout_FI			45	55					4		4	Fail
					Test	Zout_FV		20	20									
					Use-Limit	Zout_FV			45	55					5		5	Fail
					set-device									1		1		Pass

Global Specs

Global Specs

Symbol	Job	Value	Comment
Vcl_default		- 1.E+00	Detector clamp voltage low
Vch_default		6.E+00	Detector clamp voltage high

Mixed-Signal Timing Sheet

This sheet is not required because this example does not use mixed-signal timing.

Wave Definitions Sheet

This sheet is not required because this example does not define any waveforms.

< >

2015 Teradyne, Inc. All rights reserved.

gigadig_apps.5.11

< >

GigaDig Examples : Capture an HSD Clock Signal Using Undersampling : HSD Pattern with TSet and Pins

HSD Pattern with TSet and Pins

The following pattern starts the AWG6G and sources the waveform, then starts the GigaDig and captures the waveform.

TSet	Command	Label	Vector	hsd_ch1	hsd_ch2
TSet_1		start_lab…	0	1	1
TSet_1		capture_l…	1	0	0
TSet_1			2	1	1
TSet_1			3	0	0
TSet_1			4	1	1
TSet_1			5	0	0
TSet_1			6	1	1
TSet_1			7	0	0
TSet_1			8	1	1
TSet_1			9	0	0
TSet_1			10	1	1
TSet_1			11	0	0
TSet_1			12	1	1
TSet_1			13	0	0
TSet_1			14	1	1
TSet_1			15	0	0
TSet_1	jump capture_loop		16	1	1
TSet_1	halt		17	0	0

< >

gigadig_apps.5.12

< >

GigaDig Examples : Capture an HSD Clock Signal Using Undersampling : VBT Code

VBT Code

This example contains the following functions:

- Function DigitalLoopback creates a capture signal, starts the pattern, and captures the waveform, then analyzes the waveform using DSP and tests the results.
- Function CalcGigadigFsOutOfOrder calculates the sampling parameters for an out-of-order GigaDig capture.
- Functions Zoutput_FI and Zoutput_FV use the PPMUs behind the GigaDig channels to measure resistance using current and voltage.

```
Public Function DigitalLoopback() As Long
Dim pin_name As String
Dim signal_name As String
Dim pset_name As String
Dim pat_name As String
Dim datarate_bps As Double
Dim samples_per_bit As Long
Dim sample_size As Long
Dim sample_rate As Double
Dim USR As Long
Dim ooo_flag As Integer
Dim bits_per_pat As Long
Dim risetime As New SiteDouble
Dim falltime As New SiteDouble
Dim drate As New SiteDouble
Dim sampbit As New SiteDouble
pin_name = "GigaDig_Pins"
signal_name = "GigaDig_Signal"
datarate_bps = 200000000#
samples_per_bit = 2781
pset_name = "GigaDig_Pset"
pat_name = ".\HSD_Gigadig_Loopback_Digital.pat"
```

```
    bits_per_pat = 10
    sample_size = samples_per_bit * bits_per_pat
    TheHdw.Patterns.UnloadAll
    sample_rate = CalcGigadigFsOutOfOrder(samples_per_bit, _
                                datarate_bps, _
                                bits_per_pat, _
                                sample_size, _
                                USR)
    ooo_flag = 1
    TheHdw.GigaDig.Pins(pin_name).Signals.Add (signal_name)
    With TheHdw.GigaDig.Pins(pin_name).Signals(signal_name)
       .SampleRate = sample_rate
       .SampleSize = sample_size
       .VoltageRange = 2
       .BlockCount = 1
       .BlockSpacing = 0
       .ExtraSamples = 0
       If ooo_flag = 1 Then
          .UnderSamplingRatio = USR
          .ReorderedSamples = True
       End If
       .LoadSettings
    End With
    ' Apply the GigaDig signals
    TheHdw.GigaDig(pin_name).Signals(signal_name).LoadSettings
    ' Apply levels and timing and load pattern
    TheHdw.Digital.ApplyLevelsTiming True, True, True, tlPowered
    TheHdw.Patterns(pat_name).Load
    TheHdw.Patterns(pat_name).Start
    TheHdw.Wait 0.1
    TheHdw.GigaDig.Pins(pin_name).Signals(signal_name).Trigger
    ' Check that the capture hasn't timed out
    Dim timer As Boolean
    timer = TheHdw.GigaDig(pin_name).Signals.WaitForCaptureDone
    If timer = False Then
       MsgBox "Capture Timed Out"
       Stop
       Exit Function
    Else
       If TheHdw.Digital.Patgen.IsRunning Then
          TheHdw.Digital.Patgen.Halt
       End If
    End If
    ' Bind the captured data of GigaDigHD ch 1 pos to the local DSP wave
    ' The "If" allows this procedure to capture actual data when running on a tester,
    ' or to import stored sample data from a file ("Saved_waveform.txt") when running
    ' offline. You must provide the sample data in this file for the import to work.
```

```
    Dim CaptureData_1p As New DSPWave
    If TheExec.TesterMode = testModeOffline Then
        Call CaptureData_1p.FileImport("Saved_waveform.txt", File_txt)
    Else
        CaptureData_1p = TheHdw.GigaDig("Gigadig_ch1_pos").Signals(signal_name).DSPWave
    End If
    drate = datarate_bps
    sampbit = samples_per_bit
    ' Call the embedded DSP routines
    rundsp.TimeDomainAnalysis CaptureData_1p, drate, sampbit, risetime, falltime
    ' Test the results against the limits specified in the flow table
    TheExec.Flow.TestLimit resultval:=risetime, unit:=ps, forceresults:=tlForceFlow
    TheExec.Flow.TestLimit resultval:=falltime, unit:=ps, forceresults:=tlForceFlow
    On Error GoTo errhandler
    Exit Function
    errhandler:
        If AbortTest Then Exit Function Else Resume Next
    End Function
    Public Function CalcGigadigFsOutOfOrder(samples_per_bit As Long, _
                                            data_rate As Double, _
                                            data_bits As Long, _
                                            ByRef N As Long, _
                                            ByRef USR As Long) As Double
        Dim Tseff As Double ' Effective sampling resoltion = 1/Fseff
        Dim Tr As Double    ' Period of input waveform = 1 / Fr
        Dim k As Long       ' Number of effective samples to skip between actual samples
        Dim Kmin As Long
        Dim Kmax As Long
        Dim X1 As Long      ' Temp value to store number of samples when computing USR
        Dim gcd As Long     ' Greatest common divisor between two numbers
                            ' When the gcd = 1, the two numbers are relatively prime
        ' Calculate the effective sampling period:
        ' that is, the time between apparently consecutive samples
        Tseff = 1 / (data_rate * samples_per_bit)
        ' Calculate the time to burst the pattern once
        ' Tr = number of data bits in pattern * (time to burst 1 bit)
        '    = data_bits * (1/ data_rate)
        Tr = data_bits / data_rate
        ' Calculate total samples in capture
        N = data_bits * samples_per_bit
        ' Make sure this is a valid sample size
        If N > 8388552 Then
            MsgBox "Too many samples per bit, or too many bits in pattern."
            Exit Function
        End If
        ' Calculate the range for the number of effective samples to skip when capturing
        ' The minimum number of samples skipped is calculated with
```

```
    ' the maximum allowable frequency
    Kmin = 1 / (Tseff * 20000000#)
    ' The maximum number of samples skipped is calculated with
    ' the minimum allowable frequency
    Kmax = 1 / (Tseff * 10000000#)
    ' Find mutually prime values for N, USR
    k = Kmin  ' Start with minimum of samples-to-skip to achieve high sampling freq
    gcd = k
    X1 = N
    Dim R As Long ' remainder value
    Do While gcd <> 1 And k <= Kmax
      gcd = k
      R = X1 Mod k
      Do While R <> 0
        X1 = gcd
        gcd = R
        R = X1 Mod gcd
      Loop
      k = k + 1
      X1 = N
    Loop
    ' Store the return value for the undersampling ratio value
    USR = k - 1 ' decrement by 1 because of increment in last loop iteration
    ' Return the suggested actual sampling frequency
    CalcGigadigFsOutOfOrder = N / (Tr * USR)
End Function
Public Function Zoutput_FI(ppmu_pin As PinList, Itest As Double) As Long
    On Error GoTo errhandler
' Dimension objects as PinListData to contain PPMU measured and calculated results
Dim pldPPMUMeas As New PinListData
Dim pld_outR As New PinListData
Dim IRng As Double
' Set up and Connect PPMU
With TheHdw.PPMU.Pins(ppmu_pin)
    .ForceV 0
    .Connect
    .Gate = tlOn
    .ClampVHi = 4.5
    .ClampVLo = -1.2
End With
TheHdw.Wait 100 * us ' Allow settling (if needed)
' Decide current range
    IRng = 2 * ma
    If Abs(Itest) > 2 * ma Then IRng = 50 * ma
' Force current and Measure Voltage
TheHdw.PPMU.Pins(ppmu_pin).ForceI Itest, IRng
```

```
    ' Store measured Voltages in Pinlist Data
    pldPPMUMeas = TheHdw.PPMU.Pins(ppmu_pin).Read(tlPPMUReadMeasurements)
    ' Do calculations using PinListData Math
    pld_outR = pldPPMUMeas.Math.Divide(Itest)
    ' Reset and disconnect PPMU
    With TheHdw.PPMU.Pins(ppmu_pin)
        .ForceV 0
        .Gate = tlOff
        .Disconnect
    End With
    ' Datalog Results
    TheExec.Flow.TestLimit resultval:=pld_outR, _
                           ScaleType:=scaleNone, _
                           unit:=unitCustom, _
                           customUnit:="Ohm", _
                           forceVal:=Itest, _
                           forceresults:=tlForceFlow
        Exit Function
    errhandler:
        If AbortTest Then Exit Function Else Resume Next
    End Function
    Public Function Zoutput_FV(ppmu_pin As PinList, Vtest As Double) As Long
        On Error GoTo errhandler
    ' Dimension objects as PinListData to contain PPMU measured and calculated results
    Dim pldPPMUMeas As New PinListData
    Dim pld_outR As New PinListData
    Dim IRng As Double
    ' Set up and Connect PPMUs
    With TheHdw.PPMU.Pins(ppmu_pin)
        .ForceV 0
        .Connect
        .Gate = tlOn
        .ClampVHi = 4.5
        .ClampVLo = -1.2
    End With
    TheHdw.Wait 100 * us ' Wait settling (if needed)
    ' Force Voltage and Measure Current
    TheHdw.PPMU.Pins(ppmu_pin).ForceV Vtest
    ' Store measured Current in Pinlist Data
    pldPPMUMeas = TheHdw.PPMU.Pins(ppmu_pin).Read(tlPPMUReadMeasurements)
    ' Do calculations using PinListData Math
    pld_outR = pldPPMUMeas.Math.Divide(Vtest).Invert
    ' Reset and disconnect PPMUs
    With TheHdw.PPMU.Pins(ppmu_pin)
        .ForceV 0
        .Gate = tlOff
```

```
    .Disconnect
End With
' Datalog Results
TheExec.Flow.TestLimit resultval:=pld_outR, _
                       ScaleType:=scaleNone, _
                       unit:=unitCustom, _
                       customUnit:="Ohm", _
                       forceVal:=Vtest, _
                       forceresults:=tlForceFlow
Exit Function
errhandler:
    If AbortTest Then Exit Function Else Resume Next
End Function
```

< >

2015 Teradyne, Inc. All rights reserved.

gigadig_apps.5.13

< >

GigaDig Examples : Capture an HSD Clock Signal Using Undersampling : DSP Code

DSP Code

```
Public Function Roundintt(a As Double) As Long
Roundintt = Fix(a)
If Abs(a - Roundintt) > 0.5 Then
   If Roundintt >= 0 Then
      Roundintt = Roundintt + 1
   Else
      Roundintt = Roundintt - 1
   End If
End If
End Function
Public Function TimeDomainAnalysis(InputWave As DSPWave, _
                        datarate As Double, _
                        samples_per_bit As Long, _
                        retRT As Double, _
                        retFT As Double) As Long
Dim Mywave As New DSPWave, Coefficients As New DSPWave
' Smoothing algorithm
Call Coefficients.CreateConstant(1, 9, DspDouble)
Call InputWave.ConvertDataTypeTo(DspDouble)
Mywave = InputWave.FilterFIR(Coefficients, 1)
Mywave = Mywave.Select(8, 1, InputWave.SampleSize - 8).Multiply(1 / 9)
' ********** Begin Risetime Measurements **********
Dim zero_plus As Long, _
   summ As Double, _
   avg_plus As Double, _
   avg_minus As Double, _
   point_80 As Double, _
   point_20 As Double, _
   point_80_index As Long, _
   point_20_index As Long, _
   zero_minus As Long, _
   i As Long, _
   Roundint As Long, _
   j As Long, _
```

```
    slope_rt As Double, _
    slope_ft As Double
Roundint = Roundintt(0.1 * samples_per_bit)
i = 2 * samples_per_bit
' Search for the rising edge transition region
Do Until (Mywave.Element(i - 30) < 0 And _
         Mywave.Element(i) >= 0) And _
        (Mywave.Element(i - 0.5 * samples_per_bit) < 0 And _
         Mywave.Element(i + 0.5 * samples_per_bit) > 0) Or _
        (i > (Mywave.SampleSize - samples_per_bit))
   i = i + 1
Loop
zero_plus = i
' Calculate the average high voltage level
summ = 0
For i = zero_plus + CLng(samples_per_bit / 2) _
 To zero_plus + CLng(samples_per_bit / 2) + Roundint
   summ = Mywave.Element(i) + summ
Next i
avg_plus = summ / (1 + Roundint)
' Calculate the average low voltage level
summ = 0
For i = zero_plus - CLng(samples_per_bit / 2) _
 To zero_plus - CLng(samples_per_bit / 2) + Roundint
   summ = Mywave.Element(i) + summ
Next i
avg_minus = summ / (1 + Roundint)
' Find the 20% and the 80% points
point_80 = avg_plus - 0.2 * (avg_plus - avg_minus)
point_20 = avg_minus + 0.2 * (avg_plus - avg_minus)
i = zero_plus
Do While Mywave.Element(i) < point_80
   i = i + 1
Loop
point_80_index = i
i = zero_plus
Do While Mywave.Element(i) > point_20
   i = i - 1
Loop
point_20_index = i
retRT = (point_80_index - point_20_index) / (datarate * samples_per_bit)
slope_rt = (point_80 - point_20) / (point_80_index - point_20_index)
' ********** End Risetime Measurements **********
' ********** Begin Falltime Measurements **********
i = 2 * samples_per_bit
Do Until (Mywave.Element(i - 30) >= 0 And _
         Mywave.Element(i) < 0) And _
```

```
        (Mywave.Element(i - 0.5 * samples_per_bit) > 0 And _
         Mywave.Element(i + 0.5 * samples_per_bit) < 0) Or _
        (i > (Mywave.SampleSize - samples_per_bit))
    i = i + 1
Loop
zero_minus = i
summ = 0
For i = zero_minus + CLng(samples_per_bit / 2) _
 To zero_minus + CLng(samples_per_bit / 2) + Roundint
    summ = Mywave.Element(i) + summ
Next i
avg_minus = summ / (1 + Roundint)
summ = 0
For i = zero_minus - CLng(samples_per_bit / 2) _
 To zero_minus - CLng(samples_per_bit / 2) + Roundint
    summ = Mywave.Element(i) + summ
Next i
avg_plus = summ / (1 + Roundint)
point_80 = avg_plus - 0.2 * (avg_plus - avg_minus)
point_20 = avg_minus + 0.2 * (avg_plus - avg_minus)
i = zero_minus
Do While Mywave.Element(i) < point_80
    i = i - 1
Loop
point_80_index = i
i = zero_minus
Do While Mywave.Element(i) > point_20
    i = i + 1
Loop
point_20_index = i
retFT = (point_20_index - point_80_index) / (datarate * samples_per_bit)
slope_ft = (point_20 - point_80) / (point_20_index - point_80_index)
' ********** End Falltime Measurements **********
End Function
```

< >

2015 Teradyne, Inc. All rights reserved.

gigadig_apps.5.14

< >

GigaDig Examples : VBT Function Examples

VBT Function Examples

The following examples demonstrate common GigaDig tasks using VBT:

- Set Up and Capture a GigaDig Signal Using VBT
- Trigger a Capture from a Digital Pattern
- Set Up and Capture a GigaDig Signal Using a PSet
- Determine the Sample Rate for an In-Order Capture
- Determine the Sample Rate for an Out-of-Order Capture

< >

gigadig_apps.5.15

< >

GigaDig Examples : VBT Function Examples : Set Up and Capture a GigaDig Signal Using VBT

Set Up and Capture a GigaDig Signal Using VBT

This function creates a new signal, triggers a capture, and stores the capture in the signal.

```
Public Function Create_and_Capture_GigaDig_Signal(pinstr As String, _
        SignalName As String, _
        samplerate As Double, SampleSize As Long, _
        in_order As Boolean, usamp_stride As Long) As Long
  ' Reset the GigaDig to a known state
  TheHdw.GigaDig.Pins(pinstr).Reset
  ' Create the Signal with default values
  TheHdw.GigaDig.Pins(pinstr).Signals.Add (SignalName)
  With TheHdw.GigaDig.Pins(pinstr).Signals(SignalName)
    ' Use the sampling rate passed to the function
    .SampleRate = samplerate
    ' Use the sample size passed to the function
    .SampleSize = SampleSize
    ' Use the 2.048V voltage range
    .VoltageRange = 2.048
    ' Capture one block
    .BlockCount = 1
    ' Do not capture any intermediate blocks
    .BlockSpacing = 0
    ' Do not capture any extra samples before the main capture
    .ExtraSamples = 0
    ' Set the input delay to zero
    .InputDelay = 0#
    ' If the function specified out-of-order captures, set the
    ' undersampling ratio and enable reordering of captured samples
    If in_order = False Then
      .UnderSamplingRatio = usamp_stride
      .ReorderedSamples = True
    End If
    ' Program the settings
    .LoadSettings
  End With
```

```
    Dim capstr() As String
    ' Separate pin names into array elements to allow greater control
    capstr = Split(pinstr, ",", -1, vbTextCompare)
    ' Trigger the capture
    TheHdw.GigaDig.Pins(pinstr).Signals(SignalName).Trigger
End Function
```

The following function binds a DSPWave in your test procedure to the captured data. Note that this function is for a single-site test.

```
Public Function BindCapture(pinstr As String, SignalName As String, _
                            myWave As DSPWave) As Long
Set myWave = TheHdw.GigaDig.Pins(pinstr).Signals(SignalName).DSPWave.Value
End Function
```

< >

2015 Teradyne, Inc. All rights reserved.

gigadig_apps.5.16

< >

GigaDig Examples : VBT Function Examples : Trigger a Capture from a Digital Pattern

Trigger a Capture from a Digital Pattern

This function uses a pattern to trigger a GigaDig capture. Your pattern must include the following microcodes:

```
import_all_undefineds = yes;
import tset t1;
instruments = {
        SRC_CH1P:GigaDig 1;
        SRC_CH2P:GigaDig 1;
        SRC_CH3P:GigaDig 1;
        SRC_CH4P:GigaDig 1;
        SRC_CH5P:GigaDig 1;
        SRC_CH6P:GigaDig 1;
        SRC_CH7P:GigaDig 1;
        SRC_CH8P:GigaDig 1;
}
srm_vector
hsd2gdig_module ( $tset, (HSD_1, HSD_2, HSD_3, HSD_4, HSD_5, HSD_6, HSD_7, HSD_8))
{
start_label begin:
// Synchronize HSD and analog clocks
> t1     11111111                                    ;
((SRC_CH1P) = Resync)
> t1     11111111                                    ;
// Allow clock re-synchronization time of at least 450us
repeat 64000 > t1     11111111                                    ;
    .
    .
// Trigger captures on all eight GigaDig channels
((SRC_CH1P, SRC_CH2P, SRC_CH3P, SRC_CH4P, SRC_CH5P, SRC_CH6P, SRC_CH7P, SRC_CH8P) = Trig gigadig_capture)
> t1     11111111                                    ;
```

The following function sets up the HSD pattern, creates a new signal, triggers a capture from within the pattern, and stores the capture in the signal.

```
Public Function GigaDig_Trigger_From_Pattern(GDIG_pins As String, _
            SignalName As String, _
            sample_rate As Double, sample_size As Long, _
```

```
            ooo_flag As Boolean, ooo_sample_ratio As Long, _
            Pattern_Path As String) As Long
    Dim capstr() As String, i As Integer
    ' Prepare the HSD channels to use as source
    With TheHdw.Digital.Pins(HSD_pins)
        .Connect
        .InitState = chInitHi
        .StartState = chStartHi
    End With
    ' Apply the digital setup
    Call TheHdw.Digital.ApplyLevelsTiming(True, True, True)
    ' Reset the GigaDig to a known state
    TheHdw.GigaDig.Pins(pinstr).Reset
    ' Create the Signal with default values
    TheHdw.GigaDig.Pins(GDIG_pins).Signals.Add(SignalName)
    With TheHdw.GigaDig.Pins(GDIG_pins).Signals(SignalName)
        ' Use the sampling rate passed to the function
        .SampleRate = sample_rate
        ' Use the sample size passed to the function
        .SampleSize = sample_size
        ' Use the 2.048V voltage range
        .VoltageRange = 2.048
        ' Capture one block
        .BlockCount = 1
        ' Do not capture any intermediate blocks
        .BlockSpacing = 0
        ' Do not capture any extra samples before the main capture
        .ExtraSamples = 0
        ' Set the input delay to zero
        .InputDelay = 0#
        ' If the function specified out-of-order captures, set the
        ' undersampling ratio and enable reordering of captured samples
        If ooo_flag = 1 Then
            .UnderSamplingRatio = ooo_sample_ratio
            .ReorderedSamples = True
        End If
        ' Program the settings
        .LoadSettings
    End With
    ' Clear previous patterns and load the pattern passed to the function
    Patname = pattern_path
    Call TheHdw.Patterns.UnloadAll
    Call TheHdw.Patterns(Patname).Load
    ' Start the pattern
    Call TheHdw.Patterns(Patname).start("begin", "")
    ' Wait half a second for the pattern and the capture to complete
    TheHdw.Wait 0.5
```

```
    ' If the pattern and capture have not completed, raise an error
    If (TheHdw.GigaDig.Pins(GDIG_pins).IsCaptureDone = False) Then
        MsgBox ("Capture Not Done"): Exit Function
    ' ...otherwise, stop the running pattern
    Else: 'TheHdw.Digital.Patgen.Halt
    End If
    ' Split the pin list into individual items
    capstr = Split(GDIG_pins, ",", -1, vbTextCompare)
    ' Create the DSPWave to hold the captured signals
    Dim myWave() As New DSPWave
    ' Reconfigure the DSPWave array to fit the pin list passed to the function
    ReDim myWave(LBound(capstr) To UBound(capstr)) As New DSPWave
    ' Bind the captured signals to the appropriate elements of the array
    For i = LBound(capstr) To UBound(capstr)
        myWave(i) = _
    TheHdw.GigaDig.Pins(capstr(i)).Signals(SignalName).DSPWave.Value
        myWave(i).samplerate = datarate_bps * samples_per_bit
    Next i
End Function
```

< >

gigadig_apps.5.17

<	>

Set Up and Capture a GigaDig Signal Using a PSet

This function uses a PSet to configure a GigaDig, starts a pattern to apply the PSet and trigger the capture, then optionally performs DSP operations on the captured data and tests it against limits. Your pattern must include the following microcodes:

```
import_all_undefineds = yes;
import tset t3;
instruments = {
        GDIG_CH1P:GigaDig 1;
        GDIG_CH2P:GigaDig 1;
        GDIG_CH3P:GigaDig 1;
        GDIG_CH4P:GigaDig 1;
        GDIG_CH5P:GigaDig 1;
        GDIG_CH6P:GigaDig 1;
        GDIG_CH7P:GigaDig 1;
        GDIG_CH8P:GigaDig 1;
}
srm_vector
hsd2gdig_pset_module ( $tset, (HSD_1, HSD_2, HSD_3, HSD_4, HSD_5, HSD_6, HSD_7, HSD_8))
{
start_label begin3:
 > t3     11111111                                    ;
repeat 2000 > t3     11111111                                    ;
((GDIG_CH1P, GDIG_CH2P, GDIG_CH3P, GDIG_CH4P, GDIG_CH5P, GDIG_CH6P, GDIG_CH7P, GDIG_CH8P) = Enable_Inst_Cond)
 > t3     11111111                                    ;
repeat 65000 > t3     11111111                                    ;
((GDIG_CH1P, GDIG_CH2P, GDIG_CH3P, GDIG_CH4P, GDIG_CH5P, GDIG_CH6P, GDIG_CH7P, GDIG_CH8P) = PSet Pset1)
 > t3     11111111                                    ;
 > t3     11111111                                    ;
settle:
repeat 60000 > t3     11111111                                    ;
    .
    .
 > t3     11111111                                    ;
repeat 2000 > t3     11111111                                    ;
((GDIG_CH1P, GDIG_CH2P, GDIG_CH3P, GDIG_CH4P, GDIG_CH5P, GDIG_CH6P, GDIG_CH7P, GDIG_CH8P) = Trig GigaDigCapture)
```

```
 > t3    11111111                        ;
loop1:
 > t3    11111111                        ;
```

The following function uses the Add method to set up the PSet, creates a new signal, triggers a capture from within the pattern, and stores the capture in the signal.

This function takes the following parameters:

PSet_Name	The name of the PSet you want to create. This must match the name of the PSet you apply in your pattern.
PatPath	The path to your pattern that applies the PSet and triggers the capture.
All PSet parameters	For information on required PSet parameters, see Set in the VBT Syntax Reference for GigaDig.

```
Public Function GigaDig_Capture_Using_PSet(GDigPins As String, _
        PSet_Name As String, _
        PatPath As String, _
        vrange As Double, _
        sample_rate As Double, _
        sample_size As Double, _
        block_count As Double, _
        block_spacing As Double, _
        input_delay As Double, _
        extra_samples As Double, _
        ooo_flag As Boolean, _
        ooo_sample_ratio As Double) As Long
' Create a local site-aware DSPWave variable to hold the captured data
Dim Capture_Data As New DSPWave
' Prepare the HSD channels to use as source
With TheHdw.Digital.Pins(HSD_pins)
   .Connect
   .InitState = chInitHi
   .StartState = chStartHi
End With
' Apply the digital setup
Call TheHdw.Digital.ApplyLevelsTiming(True, True, True)
' Create a new PSet and assign values passed to function
With TheHdw.GigaDig.Pins(GDigPins).PSets.Add(PSet_Name)
   .VoltageRange = vrange
   .SampleRate = sample_rate
   .InputDelay = input_delay
   .BlockCount = block_count
   .BlockSpacing = block_spacing
   .ExtraSamples = extra_samples
   .SampleSize = sample_size
   If ooo_flag = True Then
      .UnderSamplingRatio = ooo_sample_ratio
      .ReorderedSamples = True
```

```
    End If
End With
' Create a signal to hold the captured data
' Note that the name of the signal matches the one that the pattern triggers
TheHdw.GigaDig.Pins(GDigPins).Signals.Add "GigaDigCapture"
' Load and start the pattern to apply the PSet and trigger the capture
Call TheHdw.Patterns(PatPath).Load
Call TheHdw.Patterns(PatPath).Start("begin3", "")
' Wait half a second for the pattern and the capture to complete
TheHdw.Wait 0.5
' If the pattern and capture have not completed, raise an error
If (TheHdw.GigaDig.Pins(GDigPins).IsCaptureDone = False) Then
    MsgBox ("Capture Not Done"): Exit Function
' Bind the DSPWave to the captured data in the Signal
Capture_Data = TheHdw.GigaDig.Pins(pinstr).Signals(SignalName).DSPWave
' Perform your DSP operations on captured data using RunDSP
' If required, test results against limits here
End Function
```

< >

2015 Teradyne, Inc. All rights reserved.

gigadig_apps.5.18

< >

GigaDig Examples : VBT Function Examples : Determine the Sample Rate for an In-Order Capture

Determine the Sample Rate for an In-Order Capture

This function calculates a valid sampling rate for in-order undersampling. It finds the highest available sampling rate for the parameters you supply.

Mixed-Signal Workshop is the preferred method of finding these values, as Solver ensures signal coherence and provides you with a range of solutions. In some cases, however, you may want to determine these parameters without using Solver.

This function assumes you are capturing a digital pattern. It takes the following parameters:

data_rate	Bit rate of the data stream in bits per second (bps).
data_bits	Number of bits in the repeatable data stream.
samples_per_bit	Number of samples to capture per bit.

and returns the following result:

N	This number is the .SampleSize parameter required when setting up a capture. This number fits into a single block (.BlockCount = 1).

The value of the function is the suggested actual (hardware) rate at which to capture samples for the waveform or pattern.

This function requires M (the number of cycles of the pattern or waveform) to be 1.

```
Public Function CalcGigadigFsInOrder(samples_per_bit As Long, _
                                     data_rate As Double, _
                                     data_bits As Long, _
                                     ByRef N As Long) As Double
Dim Fr As Double    ' Frequency of the repeating pattern = 1/ Tr
Dim Feff As Double  ' Effective sampling rate = 1/ Tseff
Dim Fs As Double    ' Actual (hardware) sampling rate = 1/ Ts
Dim K As Integer    ' Number of waveform repetitions to skip between samples
   ' Calculate time to burst one cycle of the repeating waveform
   '    Time to burst one bit = 1 / data rate
   '    Tr = time to burst 1 bit * number of bits in pattern
   '    Tr = data_bits / data_rate
   Fr = data_rate / data_bits
   ' Figure number of repetitions to skip to ensure sampling rate <= 20MHz
   K = Int(Fr / 20000000#)
```

```
    ' Calculate total number of samples to capture for entire waveform
    ' (this is a return value)
    N = (data_bits * samples_per_bit)
    ' Determine effective sampling frequency
    Feff = Fr * N
    ' Calculate actual sampling frequency so sample points advance by Feff
    ' Ts = K * Tr + Tseff
    Fs = 1# / ((K / Fr) + (1# / Feff))
    ' If actual sample rate > 20MHz, figure required number of repetitions to
    ' skip so sample rate is < 20MHz
    Do While Fs > 20000000#
        K = K + 1
        Fs = 1# / ((K / Fr) + (1# / Feff))
    Loop
    ' Return the suggested actual sampling frequency
    CalcGigadigFsInOrder = Fs
End Function
```

< >

2015 Teradyne, Inc. All rights reserved.

gigadig_apps.5.19

< >

GigaDig Examples : VBT Function Examples : Determine the Sample Rate for an Out-of-Order Capture

Determine the Sample Rate for an Out-of-Order Capture

This function calculates a valid sampling rate for out-of-order undersampling. It finds the highest available sampling rate for the parameters you supply.

Mixed-Signal Workshop is the preferred method of finding these values, as Solver ensures signal coherence and provides you with a range of solutions. In some cases, however, you may want to determine these parameters without using Solver.

This function assumes you are capturing a digital pattern. It takes the following parameters:

samples_per_bit	Number of samples to capture per bit.
data_rate	Bit rate of the data stream, in bits per second (bps).
data_bits	Number of bits in the repeatable data stream.

and returns the following results:

N	Value for the .SampleSize parameter required when setting up a capture. This number fits into a single block (.BlockCount = 1).
USR	Value for the .UnderSamplingRatio parameter required when setting up a capture.

The value of the function is the suggested actual (hardware) rate at which to capture samples for the waveform or pattern.

This function requires M (the number of cycles of the pattern or waveform) to be 1.

```
Public Function CalcGigadigFsOutOfOrder(samples_per_bit As Long, _
                                        data_rate As Double, _
                                        data_bits As Long, _
                                        ByRef N As Long, _
                                        ByRef USR As Long) As Double
Dim Tseff As Double ' Effective sampling resolution = 1/Fseff
Dim Tr As Double    ' Period of input waveform = 1 / Fr
Dim b As Long       ' Effective samples to skip between actual samples
Dim Bmin As Long
Dim Bmax As Long
Dim X1 As Long      ' Temp value to store number of samples when computing usr
Dim gcd As Long     ' Greatest common divisor (gcd) between two numbers:
                    ' When the gcd = 1, the two numbers are relatively prime
```

```
    ' Calculate the effective sampling period -
    '    that is, the time between apparently consecutive samples
    Tseff = 1 / (data_rate * samples_per_bit)
    ' Calculate the time to burst the pattern once
    ' Tr = number of data bits in pattern * (time to burst 1 bit)
    '    = data_bits * (1/ data_rate)
    Tr = data_bits / data_rate
    ' Calculate total samples in capture
    N = data_bits * samples_per_bit
    ' Make sure this is a valid sample size
    If N > 8388552 Then
       MsgBox "Too many samples per bit, or too many bits in pattern."
       Exit Function
    End If
    ' Calculate the range for the number of effective samples to skip
    ' when capturing
    ' The minimum number of samples skipped is calculated with
    ' the maximum allowable frequency
    Bmin = 1 / (Tseff * 20000000#)
    ' The maximum number of samples skipped is calculated with
    ' the minimum allowable frequency
    Bmax = 1 / (Tseff * 10000000#)
    ' Find mutually prime values for N, USR
    b = Bmin    ' Start with minimum of samples to skip
              ' This achieves high sampling frequency
    gcd = b
    X1 = N
Dim R As Long ' Remainder value
    Do While gcd <> 1 And b <= Bmax
       gcd = b
       R = X1 Mod b
       Do While R <> 0
          X1 = gcd
          gcd = R
          R = X1 Mod gcd
       Loop
       b = b + 1
       X1 = N
    Loop
    ' Store the return value for the undersampling ratio value
    USR = b - 1 ' Decrement by 1 because of increment in last loop iteration
    ' Return the suggested actual sampling frequency
    CalcGigadigFsOutOfOrder = N / (Tr * USR)
End Function
```

---

gigadig_disp.4.1

< >

GigaDig Debug Displays

GigaDig Debug Displays
GigaDig software support provides the following displays:

- GigaDigHD Board Debug Display
- GigaDig Debug Display

This section describes features specific to the GigaDig debug displays. For more information on using debug displays, see the following:

- Using Debug Displays in TDE
- Producing Code from Debug Displays

☑ **Note:**
When a test program is running, it locks the test hardware for its exclusive use. For this reason, you must stop the test program at a trap to change a parameter. If the program is running (or waiting for input), you cannot change the display until the test program halts or is aborted.

< >

gigadig_disp.4.2

< >

GigaDig Debug Displays : GigaDigHD Board Debug Display

GigaDigHD Board Debug Display

The GigaDigHD Board Debug Display is a board specific display showing conditions on a single GigaDigHD board. The display provides an overview of instrument connections and lets you set some boardwide parameters.

< >

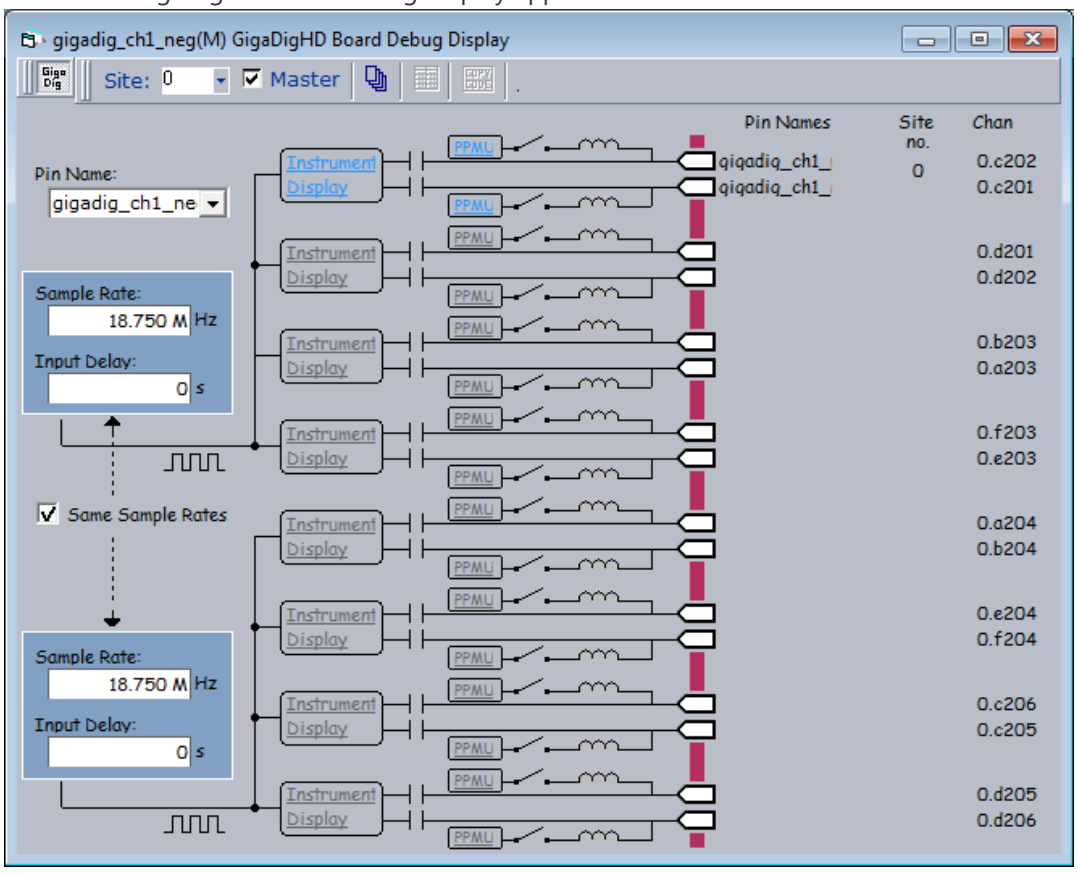
gigadig_disp.4.3

< >

GigaDig Debug Displays : GigaDigHD Board Debug Display : Starting the GigaDigHD Board Debug Display

Starting the GigaDigHD Board Debug Display

To launch the GigaDigHD Board Debug Display, on the IG-XL Tools tab, in the Launch group, select Subsystems>GigaDigHD Board. The GigaDigHD Board Debug Display appears.



GigaDigHD Board Debug Display

Using the Display

As you run a test, the debug display changes to show the state of the GigaDig board at the current position in the test program. Channel information appears for each instrument and relays open and close as the state of the instrument changes. Color-coded lines highlight the signal path.

The display does not contain information until you load and validate a test program.

The display includes the following controls:

Chan	Specifies the DIB channels to which the instrument connects for the current site.

Site no.	Specifies the site or sites to which each instrument connects. If an instrument connects to multiple sites, you can select a different site from the list.
Pin Names	Specifies the pin or pins to which each instrument connects. If an instrument connects to multiple pins, you can select a different pin from the list.
PPMU	 Opens the PPMU debug display opens and shows the parameters for the selected channel. If the test program does not contain a GigaDig pin for a specific channel, the link for that channel is disabled.
Instrument Display	Opens the GigaDig Debug Display for the selected instrument. If the test program does not contain a GigaDig pin (Pos or Neg) for that channel, the link for that channel is disabled.
Sample Rate	Sets the sample rate. Valid values are from 10.000MHz to 20.000MHz. You can set one sample rate per board; changing the value in one Sample Rate box changes the value in the other.
Input Delay	Sets the input delay. Valid values are from 0s to 10.000s. You can set one input delay per board; changing the value in one Input Delay box changes the value in the other.
Pin Name	Lists all GigaDig pins. When you select one, the display shows you the entire board that contains that pin. If you select a pin on a different board, the display changes to show you the new board.

< >

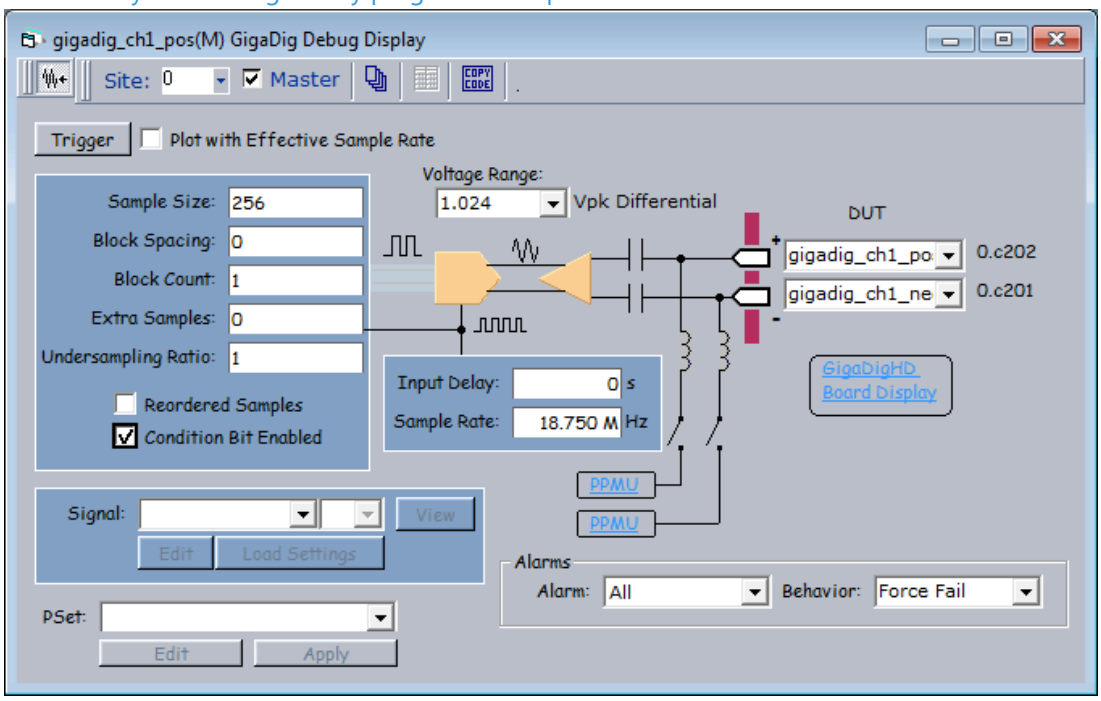
gigadig_disp.4.4

< >

GigaDig Debug Displays : GigaDig Debug Display

GigaDig Debug Display

The GigaDig Debug Display is an interactive display that shows a block diagram of the pin electronics in the context of a site and allows you to change many programmable parameters.



GigaDig Debug Display

This section describes features specific to the GigaDig debug displays and includes the following information:

- Starting the GigaDig Instrument Debug Display
- GigaDig Debug Display Interface
- Making Channel Connections Using the GigaDig Debug Display

For other features common to all debug displays, see Using Debug Displays in TDE in the TDE section. These features include storing and comparing hardware states and the use of the Summary Display for each instrument.

< >

gigadig_disp.4.5

< >

GigaDig Debug Displays : GigaDig Debug Display : Starting the GigaDig Instrument Debug Display

Starting the GigaDig Instrument Debug Display

To open the GigaDig Debug Display, on the IG-XL Tools tab, in the Launch group, select Debug Displays>GigaDig.

The display does not contain information until you load and validate a test program.

< >

gigadig_disp.4.6

< >

GigaDig Debug Displays : GigaDig Debug Display : GigaDig Debug Display Interface

GigaDig Debug Display Interface

The GigaDig debug display includes the following sets of controls:

- Signal Properties
- PSet Properties
- Alarm Properties
- Other Controls

Signal Properties

The GigaDig debug display includes the following controls that show and let you change the parameters of the capture.

GigaDig Debug Display Signal Controls

Parameter/Control	Description
Condition Bit Enabled	Instructs the capture instrument to generate a condition bit when the capture is complete. See Condition Bit Programming.
Trigger	Triggers a capture using the settings from the debug display. This functions similarly to a single trigger button on an oscilloscope. To capture data, set the capture's parameters (sample rate, block count, block size, block spacing, and extra samples) on the display, then click Trigger. The trigger is disassociated from the Signal name. No signal must exist for the Trigger button to function. Captures made from the display are shown on WaveScope, and the data is then discarded. So data are for visual reference only. You cannot use data from the debug display capture for processing because the captured samples are not moved through Instrument Initiated Move (IIM) to the Support Board. Also, the debug display does not store the data. Once you have displayed it, therefore, you cannot retrieve it.
Plot with Effective Sample Rate	Uses the effective sample rate when triggering a capture.

Sample Size	Sets the number of samples per block. Minimum size: 128 Maximum size: 8 388 552 See Capture Sample Size Parameters.
Block Spacing	Sets the number of samples to skip per block when the block count is greater than 1. Minimum number of samples to skip: 0 Maximum number of samples to skip: 16 777 215 See Capture Sample Size Parameters.
Block Count	Sets the number of blocks to capture. Minimum number of blocks: 1 Maximum number of blocks: 65 536 See Capture Sample Size Parameters.
Extra Samples	Sets the number of extra samples per block. These extra samples appear at the beginning of each block. Minimum extra samples per block: 0 Maximum extra samples per block: 8 388 296 See Capture Sample Size Parameters.
Undersampling Ratio	Sets the undersampling ratio. If you enable Reordered Samples, the DSP preprocessor reorders the samples according to this ratio. Minimum ratio: 1 Maximum ratio: 16 777 215
Reordered Samples	Enables reordering of samples during DSP preprocessing. If you check the box, IG-XL rearranges the captured samples according to the undersampling ratio.
Signal	Selects a signal from those you have defined in VBT. When IG-XL moves a capture to the Support Board and does not redirect it to a DSP Procedure, the signal's name appears with an asterisk (*). You can view such a signal in WaveScope. When multiple captures on the Support Board are associated with the same Signal name, a list box to the right of this control allows you to selects which captures you want to view.
View	Opens WaveScope to view the selected signal. Viewing a signal makes a copy of the samples on the DSP computer; it does not remove the data from the DSP computer. If the signal contains multiple captures, use the list box to the right of the Signal control to select which capture to view.
Edit	Brings up the PSet and Signals Editor (PSSE) with the selected signal loaded. This control is grayed out if no signals are available. See PSet and Signals Editor in the TDE section.
Load Settings	Loads the settings of the specified Signal. You must first choose a signal from the Signal list.
Voltage Range	Sets the voltage range of the capture. Choose the appropriate value from the list.
Input Delay	Sets the adjustment to the phase of the clock. Note that this applies to all channels in the group; that is, channels 1 through 4 share a value for this parameter, and

	channels 5 through 8 share a separate value for this parameter. Minimum delay: 0s Maximum delay: 10.000s
Sample Rate	Sets the number of samples to capture per second. Note that this applies to all channels in the group; that is, channels 1 through 4 share one value for this parameter, and channels 5 through 8 share a second value for this parameter. Minimum rate: 10.000MHz Maximum rate: 20.000MHz

PSet Properties

The GigaDig debug display includes the following controls that show and let you load and apply a parameter set (PSet).

GigaDig Debug Display PSet Controls

Control	Description
PSet	Lists the parameter sets (PSets) defined in VBT. To apply a PSet from the debug display, you must have previously defined PSets using VBT. Otherwise, the list is empty.
Edit	Brings up the PSet and Signals Editor (PSSE) with the selected PSet loaded. This control is grayed out if no PSets are available. See PSet and Signals Editor in the TDE section.
Apply	Allows you to apply a PSet from the PSet list. This control is grayed out if no PSets are available.

Alarm Properties

The GigaDig debug display includes the following controls that show and let you control the behavior of alarms originating from the GigaDig.

GigaDig Debug Display Alarm Controls

Control	Description
Alarm	Selects the alarm for which to set the Behavior. The options are: • All • Over Range Note that All and Over Range perform the same function. Also, the special option All is write-only, so the Behavior displayed for it is not updated if it is changed using VBT.
Behavior	Selects the action that should occur if the selected alarm condition exists for the pin. The options are: • Continue • Default

- Force Bin
- Force Fail
- Off

Other Controls

The GigaDig debug display includes the following additional controls.

GigaDig Debug Display Additional Controls

Control	Description
DUT	Shows the pin or pins connected to each channel. Generally this is a pos/neg pair. If the channel is connected to multiple pins, you can show information for that pin by choosing the pin's name from the list. This also shows the DIB channel to which the instrument connects.
GigaDigHD Board Display	Opens the GigaDigHD Board Debug Display for the board that contains the instrument you are viewing.
PPMU	Opens the PPMU debug display so you can view and edit measurement settings for the channel.

gigadig_disp.4.7

< >

GigaDig Debug Displays : GigaDig Debug Display : Making Channel Connections Using the GigaDig Debug Display

Making Channel Connections Using the GigaDig Debug Display

You can change a connection in the following ways:

- To open or close a relay, click the relay.

The relay changes state and appears in bold.

- To connect to a different pin, click the appropriate box in the Pin Names column and select a pin from the list.

< >

gigadig_prog.3.01

< >

GigaDig Programming

GigaDig Programming

This section describes programming of a GigaDig to capture data from a device.

The following topics are available:

- GigaDig Programming Concepts
- Programming Overview
- GigaDig DataTool Programming
- Determining Sampling Parameters
- GigaDig Pattern Control
- Using GigaDig in Concurrent Testing

For more information, see:

- GigaDig (VBT Syntax Reference)
- GigaDig Examples

< >

gigadig_prog.3.02

< >

GigaDig Programming : GigaDig Programming Concepts

GigaDig Programming Concepts

This section discusses the following programming concepts:

< >

gigadig_prog.3.03

< >

GigaDig Programming : GigaDig Programming Concepts : Simulated Configuration Files

Simulated Configuration Files

You can use the GigaDig offline by forcing IG-XL to map the GigaDig. Do this by creating a SimulatedConfig.txt file and placing the file in any of the following locations:

- Folder containing the IG-XL workbook
- \tester folder: C:\Program Files (x86)Teradyne\IG-XL\<version>\tester
- \bin folder: C:\Program Files (x86)\Teradyne\IG-XL\<version>\bin

The following SimulatedConfig.txt file maps the GigaDig board in slot 3:

```
<Version>
1
</Version>

<TEST_HEAD TH 1>
#slot[.subslot] Type          idprom (type serial rev company)

-1.0       dib              239-001-00 0000000 0000-A 0000

1.0        HSD-M            979-219-00 0000000 0000-A 0000
           HSD-M            956-361-00 0000000 0000-A 0000

6.0        DC-30            805-002-00 0000000 0000-A 0000
           DC-30            949-901-00 0000000 0000-A 0000

3.0        GigaDigHD        979-999-00 0000000 0000-A 0000
           GigaDigHD        956-436-00 0000000 0000-A 0000

12.0       BBAC-15          979-214-00 0000000 0000-A 0000
           BBAC-15          956-375-00 0000000 0000-A 0000

24.0       SupportBoard     974-220-12 0000000 0000-A 0000
           SupportBoard     956-354-00 0000000 0000-A 0000
```

</TEST_HEAD TH 1> t

To use the GigaDig offline, the SimulatedConfig.txt file must include every board needed in the IG-XL workbook. The SimulatedConfig.txt file represents the state of the offline tester. To use the GigaDig online, you need not edit any files. IG-XL automatically maps all boards in the tester.

For more information, see Tester Configuration.

< >

2015 Teradyne, Inc. All rights reserved.

gigadig_prog.3.04

< >

GigaDig Programming : GigaDig Programming Concepts : Capture Sample Size Parameters

Capture Sample Size Parameters

You use the GigaDig to sample data from a waveform or data set that repeats at known intervals. To ensure that you capture the correct information, you must tell the GigaDig where to find useful information. You do this by setting capture sample parameters.

The SampleSize parameter specifies the number of samples to capture. This must be at least 128. The default value is 256.

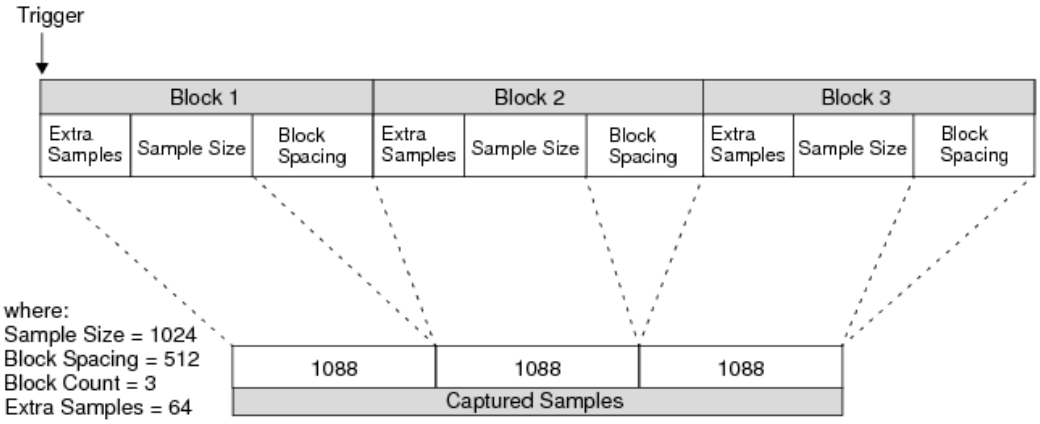
To capture additional samples before the data of interest, you can specify how many of these samples to capture using the ExtraSamples parameter. This can help you measure additional signal attributes, or you can use the time to manage expected behavior such as device settling times. The GigaDig stores these samples in capture memory. Unless you want to capture only part of a recurring waveform, such as the data segment of a network packet, this parameter is useful generally only when BlockCount is 1 and BlockSpacing is 0. You do not need to capture any extra samples; the default value for this parameter is 0.

If you are interested in the information in a specific part of each repeating set of data, use the BlockSpacing parameter to direct the GigaDig to ignore a number of samples at the end of each block set. The GigaDig skips these samples and does not store them in capture memory. You do not need to skip samples; the default value for this parameter is 0.

The BlockCount parameter specifies how many blocks to capture. This must be at least 1, which is the default value.

The total capture size, in samples, is (SampleSize + ExtraSamples) * BlockCount.

The following figure shows the relation of the sample size, extra samples, and block spacing.



Capture Sample Size Parameters

For information on using VBT to specify each of these parameters, see:

- TheHdw.GigaDig.Pins.ExtraSamples
- TheHdw.GigaDig.Pins.SampleSize

- TheHdw.GigaDig.Pins.BlockSpacing

- TheHdw.GigaDig.Pins.BlockCount

< >

gigadig_prog.3.05

< >

GigaDig Programming : GigaDig Programming Concepts : Ordering Captured Samples

Ordering Captured Samples

Undersampling is a technique that allows you to sample waveforms with a frequency higher than the maximum sampling frequency of the hardware. This technique assumes that the data repeats periodically and is of a fixed length. You can take samples from multiple repetitions of a high frequency signal at a slower sample rate, and then use those samples to rebuild the higher frequency signal.

The GigaDig supports both Capturing Data In Order and Capturing Data Out of Order, and it can automatically rearrange out-of-order samples to provide a correct representation of the dataset.

Use in-order capture when the entire period between samples (that is, the length of the repeating pattern plus one resolution step) is greater than the GigaDig minimum sampling frequency (10MHz).

Use out-of-order capture when:

- The period between samples is less than 10MHz
- You want to minimize test time

Capturing Data In Order

The GigaDig performs asynchronous captures; that is, capturing does not necessarily start at the beginning of a waveform. Therefore, you must know the length of the block of data to be able to capture a sufficient number of samples to allow you to reconstruct the waveform. The sampling rate is:

(1) $N + \Delta$

where:

N	Is an integer multiple of period of the block of data.
Δ	Is one resolution step such that N*Δ equals the period of the repeating waveform.

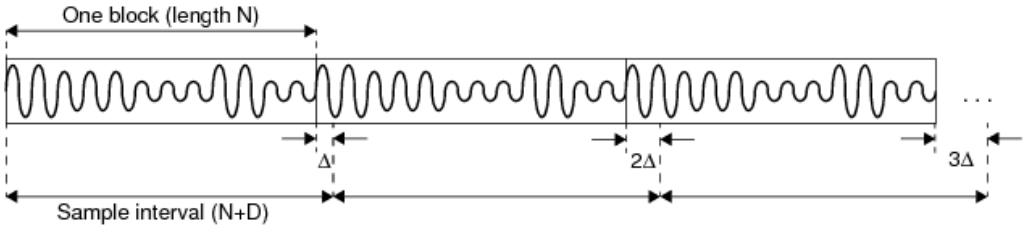
Δ must fit into N an integer number of times. For a description, see Determining the Sampling Rate.

To capture data using in-order undersampling:

1. Determine a sampling rate.
2. Sample the waveform at that rate.
3. Continue sampling at this fixed interval until you capture N samples.

You have now captured sufficient samples to represent your waveform or data set.

The following figure shows the progression of samples using in-order undersampling.



In-Order Undersampling

This produces an effective sampling rate of 1/Δ, or a sample with a resolution of Δ.

For the GigaDig, the total sample period (N+Δ) must be between 10MHz and 20MHz.

Capturing Data Out of Order

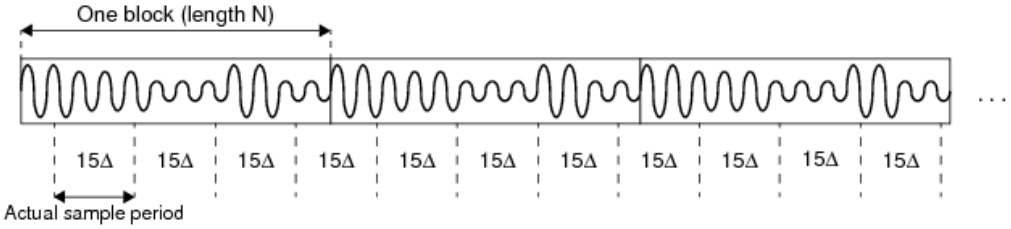
Out-of-order undersampling allows you to collect multiple samples from each block, or repetition, of data. This means you can collect enough samples to reconstruct a waveform from fewer repetitions of the data, shortening test time. For a description of how to determine the sample rate for out-of-order undersampling, see Determining Sampling Parameters.

You can use out-of-order undersampling when there is no coherent timing solution for in-order undersampling or when trying to minimize test time.

To capture data using out-of-order undersampling:

1.	Determine a sampling rate.
2.	Sample at one point in a block of data.
3.	Continue to sample at this rate until you capture N samples.

The following figure shows the progression of samples using out-of-order undersampling.



Out-of-Order Undersampling

To maintain an effective sampling rate of 1/Δ, you tell IG-XL how many Δ's appear between captures (15 in this figure). This is called the undersampling ratio and is equal to the ratio of the effective sample rate (Fs_{eff}) to the actual hardware sample rate (Fs_{actual}). IG-XL can then automatically reorder the samples according to the formula:

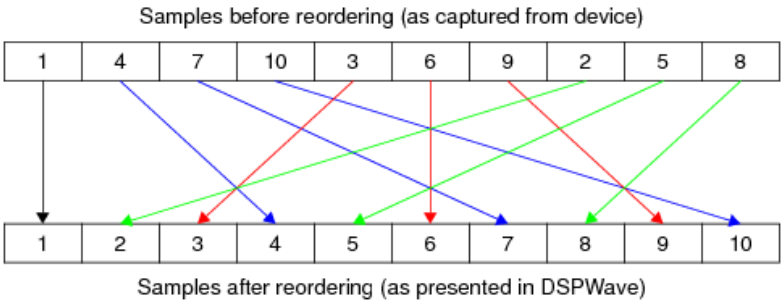
(index * undersampling_ratio) modulo N

where:

index	Is the offset from the first sample in the block.
N	Is the number of samples in the block.

This rearranges the samples so they appear in an order that correctly describes the waveform. This happens before any DSP procedures, so your procedures can work with data that correctly represents the waveform from the device.

The following figure shows a ten-sample capture with an undersampling ratio of 3.



Reordering of Samples from Out-of-Order Capture

For the GigaDig, the total sample period must be between 10MHz and 20MHz.

For information on programming out-of-order undersampling using VBT, see:

- TheHdw.GigaDig.Pins.ReorderedSamples
- TheHdw.GigaDig.Pins.UnderSamplingRatio

< >

gigadig_prog.3.06

< >

GigaDig Programming : GigaDig Programming Concepts : Condition Bit Programming

Condition Bit Programming

The GigaDig provides a condition bit that you can use to control branching in a digital pattern. This allows a pattern to enter a capture loop so that it can keep running (producing DUT clocks, digital I/O, and so on) until the capture completes. When the capture completes, the pattern can either halt or set a flag to indicate to the host computer that the capture is finished. You can then suspend execution of the test program until the capture completes, at which time it is safe to set up the next test.

This use of the condition bit allows for flexible capture sizes and provides a reliable way to determine when the digitizer completes its capture.

Each GigaDig channel can have its own microcode column in a pattern. Using the Enable_Inst_Cond and Disable_Inst_Cond microcodes, you can open and close a window that allows the end of capture instrument condition pulse to move to the pattern generator. This allows you to select dynamically which instrument delivers a condition bit during a pattern burst. Note that if the window is not open at the time when the capture completes, the pulse does not reach the pattern generator.

In addition to pattern microcode control of the GigaDig condition bit, you use the following VBT code to control enabling and disabling the bit on the channel:

```
' Enable the capture instrument condition bit:
TheHdw.GigaDig.Pins(CapPin).ConditionBitEnabled = True
' Disable the capture instrument condition bit:
TheHdw.GigaDig.Pins(CapPin).ConditionBitEnabled = False
```

For the GigaDig to set the condition bit, you must do both of the following:

- Use the ConditionBitEnabled VBT statement to enable the bit on that channel
- Use Enable_Inst_Cond microcode to the enable the window

IG-XL OR's all UltraFLEX test system instrument condition bits before sending them to the pattern generator. Therefore, you typically enable the condition bit window for one instrument at a time to make sure that you know which instrument has completed its capture or reached another state that sets its condition bit.

If the instrument is not under pattern control and you are not concerned with precise timing, you can use either of the following VBT syntax to notify you when a capture completes:

TheHdw.GigaDig.Pins.Signals.WaitForCaptureDone

TheHdw.GigaDig.Pins.IsCaptureDone

For more information, see About PatternTool.

< >

gigadig_prog.3.07

< >

GigaDig Programming : GigaDig Programming Concepts : GigaDig Alarms

GigaDig Alarms

An alarm occurs when a sample being moved from capture memory is at or outside the limits of the current voltage range. The GigaDig records alarms for the first signal per instrument. If you capture multiple signals per instrument during a single test, the report notes only those alarms that occurred during the first signal. The instrument records the total number of alarms that occurred during the capture of a specific signal, but does not record which specific samples caused the alarm.

For each signal that causes alarms, IG-XL returns a message in the Test Output window similar to the following:

[7.c102, Site 1] GDIHD:0010 : The GigaDig over-ranged for 123 samples during the capture that was triggered with the following signal name: "bar".

An alarm does not halt the test program.

The syntax for masking and enabling alarms and action to take if an alarm occurs is as follows:

- For all channels on a GigaDig instrument: TheHdw.GigaDig.Alarm
- For specific GigaDig pins: TheHdw.GigaDig.Pins.Alarm

< >

gigadig_prog.3.08

< >

GigaDig Programming : GigaDig Programming Concepts : Error Reporting

Error Reporting

IG-XL returns GigaDig hardware errors to the Test Output window. These consist of invalid hardware conditions, such as executing two microcodes too close together. They are not safety-related and do not represent a risk of hardware damage; they do not halt the test program. Instead, they alert you to a condition you cannot ignore. The errors indicate that your test program has attempted to do something that the hardware cannot do.

If this type of error occurs repeatably, you must resolve the error before you release your test program into production.

If the error occurs only occasionally, do not stop a test program in production mode. Instead, your test program should return the error and fail the current device but continue production. During test program debug, you may choose to halt execution to help you determine the cause of the condition.

When a site fails, the site becomes inactive, and IG-XL no longer programs any instruments unique to that site. For multisite test programs, the GigaDig continues capturing data from any active site from which it expects input.

< >

gigadig_prog.3.09

< >

GigaDig Programming : GigaDig Programming Concepts : GigaDig Connections

GigaDig Connections

Because the inclusion of relays in the signal path imposes bandwidth limitations, the GigaDig does not use DUT relays and cannot be the primary instrument in a DIB Access line. GigaDig inputs are always connected directly to the DUT.

However, the GigaDig can be the secondary instrument behind a DIB Access line. Therefore, the .Connect and .Disconnect methods control the DIB Access relays on the primary instrument.

For more information, see:

- Multiple Resources and DIB Access
- DIB Access Traces
- TheHdw.GigaDig.Pins.Connect
- TheHdw.GigaDig.Pins.Disconnect

< >

gigadig_prog.3.10

< >

GigaDig Programming : GigaDig Programming Concepts : Single-Ended Use

Single-Ended Use

IG-XL programs both sides of a GigaDig differential channel whether you specify the positive pin, the negative pin, or both pins. Even when you perform a single-ended capture by sourcing a signal to GigaDigPos or GigaDigNeg only, both pins are active. To make a single-ended capture, tie one end to ground on the DIB through 50ohms as close as possible to the GigaDig input.

< >

gigadig_prog.3.11

< >

GigaDig Programming : GigaDig Programming Concepts : GigaDig Reset Values

GigaDig Reset Values

The following table lists the reset values for the GigaDig.

GigaDigHD Reset Values

Setting	Default Value
All alarms	Enabled
BlockCount	1
BlockSpacing	0
Condition bit	Enabled
ExtraSamples	0
InputDelay	0.0s
Reordering of samples	Disabled
SampleRate	18.75MHz
SampleSize	256
Undersampling ratio	1 (no out-of-order undersampling)
Voltage range	1.024V peak differential

Resetting a single channel can affect the settings for other channels with shared properties. For example, since the input delay property is common to all channels on a GigaDig board, setting the input delay for one channel also sets the property for all other channels on the same board.

< >

gigadig_prog.3.12

< >

GigaDig Programming : Programming Overview

Programming Overview

To develop a test program that uses the GigaDig instrument, follow this basic procedure:

1.	Create a pin map to describe tester-to-device pin connections.

See Programming the Pin Map.

2.	Create a channel map to describe channel connections.

See Programming the Channel Map.

3.	Determine the parameters of the measurement.

See Determining Sampling Parameters.

4.	Include GigaDig microcodes in your digital pattern. (This step is optional.)

See GigaDig Pattern Control.

5.	Create a test procedure to perform your tests.

6.	Add your tests to the Flow Table.

7.	Validate and run your test program.

VBT syntax is described in the GigaDig section of the VBT Syntax Reference.

< >

gigadig_prog.3.13

< >

GigaDig Programming : GigaDig DataTool Programming

GigaDig DataTool Programming

This section describes how to program several DataTool worksheets to handle basic GigaDig parameters. It includes the following topics:

< >

gigadig_prog.3.14

< >

GigaDig Programming : GigaDig DataTool Programming : Programming the Pin Map

Programming the Pin Map

A pin map describes the device pins by assigning a pin type to each device pin name. Select the Analog pin type for each device pin name connected to a GigaDig instrument.

Programming settings for either the Pos or Neg pin of a differential pin pair programs both pins of the pair. You do not need to program the pins individually or to create a differential pin group. However, programming both pins does not cause problems.

For more information, see Pin Map Sheet in the DataTool section.

< >

gigadig_prog.3.15

< >

GigaDig Programming : GigaDig DataTool Programming : Programming the Channel Map

Programming the Channel Map

A Channel Map sheet assigns individual device pin names to test system channels and power supplies. For more information, see Channel Map Sheet in the DataTool section.

For the GigaDig, valid channel types are:

GigaDigNeg	GigaDig Negative Pin
GigaDigPos	GigaDig Positive Pin

Because the GigaDig is a differential instrument, the “positive” input carries the signal and the “negative” input carries the inverse of the signal.

Type the tester channel information value for each site to connect a DUT pin to the instrument. The following table shows the channel map format for GigaDig channels (where x indicates the DIB slot number).

GigaDig Channel Resources

GigaDig Pogo Pin	DIB Channel	Signal Name
GigaDig Capture Pos 1	x.c202	x.cappos1
GigaDig Capture Neg 1	x.c201	x.capneg1
GigaDig Capture Pos 8	x.d205	x.cappos8
GigaDig Capture Neg 8	x.d206	x.capneg8

You can program a particular GigaDig instrument by specifying either the positive or negative pin or channel on that instrument. All GigaDig language elements apply to both channels, so you do not need to create differential pin groups containing a positive and a negative channel from the same differential pair. If you do program both the positive and negative pins of a differential pair, no negative effects occur.

To use the PPMUs behind both positive and negative channels, you must explicitly define both GigaDigPos and GigaDigNeg pins of a differential channel in the channel map. During validation, IG-XL verifies that each pin definition is correct (that is, the positive pin at SlotX.ChannelY is of type GigaDigPos and the negative pin at SlotX.ChannelZ is of type GigaDigNeg.)

For more information, see Specifying Tester Channels in the DataTool section.

Sharing GigaDig Resources

You can share a single GigaDig instrument among multiple sites or pins on the same site using standard IG-XL and UltraFLEX procedures.

If you use a relay to modify the connection between a single GigaDig resource and multiple sites or pins on a DIB, you must use utility bits or other standard DIB control mechanisms to control the relay. There is no GigaDig language to change the state of a relay.

Using One Resource for Multiple Sites

To share a single resource across multiple sites:

1. Define a differential pin pair for Site 0 in the channel map as normal.
2. For Site 1 and each subsequent site that uses the same pin pair, specify Site0 in the Tester Channel column for that site.

For example:

Channel Map Entries for GigaDig Pins Shared Across Sites

Pin Name	Type	Site 0	Site 1	Site 2
GDPos1	GigaDigPos	1.cappos1	Site0	Site0
GDNeg1	GigaDigNeg	1.capneg1	Site0	Site0

For more information, see Sharing Across Sites in the DataTool section.

Using One Resource for Multiple Pins

To share a single resource across multiple pins on the same site:

1. Define a differential pin pair for each pin in the channel map as normal.
2. For the first instance of the resource, declare the pin pair in the Site column as normal.
3. For the second and subsequent instances of each pin, declare the pin in the Site column as S: followed by the pin name from the first instance.

For example:

Channel Map Entries for GigaDig Pins Shared Across Pins

Pin Name	Type	Site 0
GDPos1	GigaDigPos	1.cappos1
GDNeg1	GigaDigNeg	1.capneg1
GDPos2	GigaDigPos	S:GDPos1
GDNeg2	GigaDigNeg	S:GDNeg1

For more information, see Sharing Between Pins in the DataTool section.

Using Multiple Resources Behind One Pin

To use the GigaDig as the primary resource behind a DUT pin that connects to multiple resources:

1. Define a differential pin pair for each GigaDig pin in the channel map as normal.

2. For the GigaDig, declare the pin pair in the Site column as normal.

3. For the second and subsequent resources behind the pin:

   a. Declare the pin in the Pin Name column using the GigaDig pin name followed by :M.

   b. Fill the Type column using the correct type for the instrument.

   c. Fill each Site column with the channel location for the instrument.

For example:

Channel Map Entries for GigaDig Pins Shared Across Pins

Pin Name	Type	Site 0	Comments
GDPos1	GigaDigPos	1.cappos1	Pos channel from GigaDig
GDNeg1	GigaDigNeg	1.capneg1	Neg channel from GigaDig
GDPos1:M	AWGPos	3.srcpos1	Pos channel from AWG (secondary)
GDNeg1:M	AWGNeg	3.srcneg1	Neg channel from AWG (secondary)

To use the GigaDig as a secondary resource behind DUT pin that connects to multiple resources:

1. Define each shared pin in the channel map as normal.

2. For the GigaDig:

   a. Declare the pin in the Pin Name column using the original pin name followed by :M.

   b. Select GigaDigPos or GigaDigNeg from the Type column.

   c. Fill the Site column with the channel location for the GigaDig channel.

For example:

Channel Map Entries for GigaDig Pins Shared Across Pins

Pin Name	Type	Site 0	Comments
AWGPos1	AWGPos	3.srcpos1	Pos channel from AWG
AWGNeg1	AWGNeg	3.srcneg1	Neg channel from AWG
AWGPos:M	GigaDigPos	1.cappos1	Pos channel from GigaDig (secondary)
AWGNeg1:M	GigaDigNeg	1.capneg1	Neg channel from GigaDig (secondary)

For more information, see Multiple Resources in the DataTool section.

Using the GigaDig with DIB Access

You can use the GigaDig as a secondary instrument in a DIB access connection. As there is no DIB access path on the GigaDig instrument, you cannot use the GigaDig as the primary instrument in a DIB access connection. Attempting to do so produces a validation error.

For more information, see DIB Access in the DataTool section.

< >

2015 Teradyne, Inc. All rights reserved.

gigadig_prog.3.16

< >

GigaDig Programming : Determining Sampling Parameters

Determining Sampling Parameters

The major function of the GigaDig is to capture analog waveforms from a device. The GigaDig uses timing sets to specify capture sampling parameters such as sample rate and sample size.

You can calculate sampling parameters in either of the following ways:

- Creating a Timing Set Using the Mixed Signal Workshop (the preferred method)
- Determine Sampling Parameters Independently

< >

gigadig_prog.3.17

< >

GigaDig Programming : Determining Sampling Parameters : Creating a Timing Set Using the Mixed Signal Workshop

Creating a Timing Set Using the Mixed Signal Workshop

The Mixed Signal Workshop (MSW) tool helps you to build reusable mixed signal timing solutions that meet coherence requirements.

The WaveDesigner tool creates wave definitions that are independent of timing and instrument specific information. The MSW uses these wave definitions and applies the necessary timing information to them to generate samples.

The output of the MSW is a mixed signal timing set that contains the signal's structural characteristics and the details of the clocking solution that you created. Timing sets are stored on a Mixed Signal Timing sheet in the IG-XL workbook and become a reusable timing solution for future tests.

To create the required timing set:

1. Insert and set up a Mixed Signal Timing sheet in the IG-XL workbook:
   a. Select Insert>Insert IG-XL Worksheet in the Sheet group on the IG-XL Main tab.
   b. Select Mixed Signal Timing from the Sheet type menu in the Insert IG-XL Worksheet dialog box.
   c. Type a name for the sheet in the Sheet name box (optional). The default name is Mixed Signal Timing.
   d. Click OK.
   e. Type a timing set name in the Set Name column and press Enter.
   f. Right-click the timing set name and select Edit from the context menu to start the Mixed Signal Workshop (MSW) tool in TDE.

The MSW display appears and displays the DUT and capture resources blocks.

2. Add and configure a source instrument.
3. Add a GigaDig resource to the MSW display:
   a. Select MSW>Add Capture.

b. Select GigaDig from the Select Resource dialog box.

Leave the Resource ID box empty unless this timing set uses additional GigaDig instruments. If the timing set does use additional instruments, assign the current source instrument a unique ID number to distinguish it from the others.

c. Click OK in the Select Resource dialog box.

The GigaDig resource block appears on the MSW display.

4. Specify Capture timing.

Mixed Signal Workshop's Solver provides the most efficient and flexible way of finding timing solutions and is therefore the preferred method of determining the sampling parameters. GigaDig programming uses a modified Solver. For more information, see Determining Sampling Parameters Using the MSW Solver.
To determine capture and undersampling parameters without using Solver, see Determine Sampling Parameters Independently.

5. Click Save to Excel in the toolbar to save the timing set.

For more information, see Mixed Signal Workshop.

< >

2015 Teradyne, Inc. All rights reserved.

gigadig_prog.3.18

< >

GigaDig Programming : Determining Sampling Parameters : Determining Sampling Parameters Using the MSW Solver

Determining Sampling Parameters Using the MSW Solver

Using the Mixed Signal Workshop Solver for GigaDig helps you to create a list of all possible timing solutions that satisfy the frequency target values, constraints, instrument restrictions, and undersampling method. The standard Solver provides solutions for instruments that sample in real time.

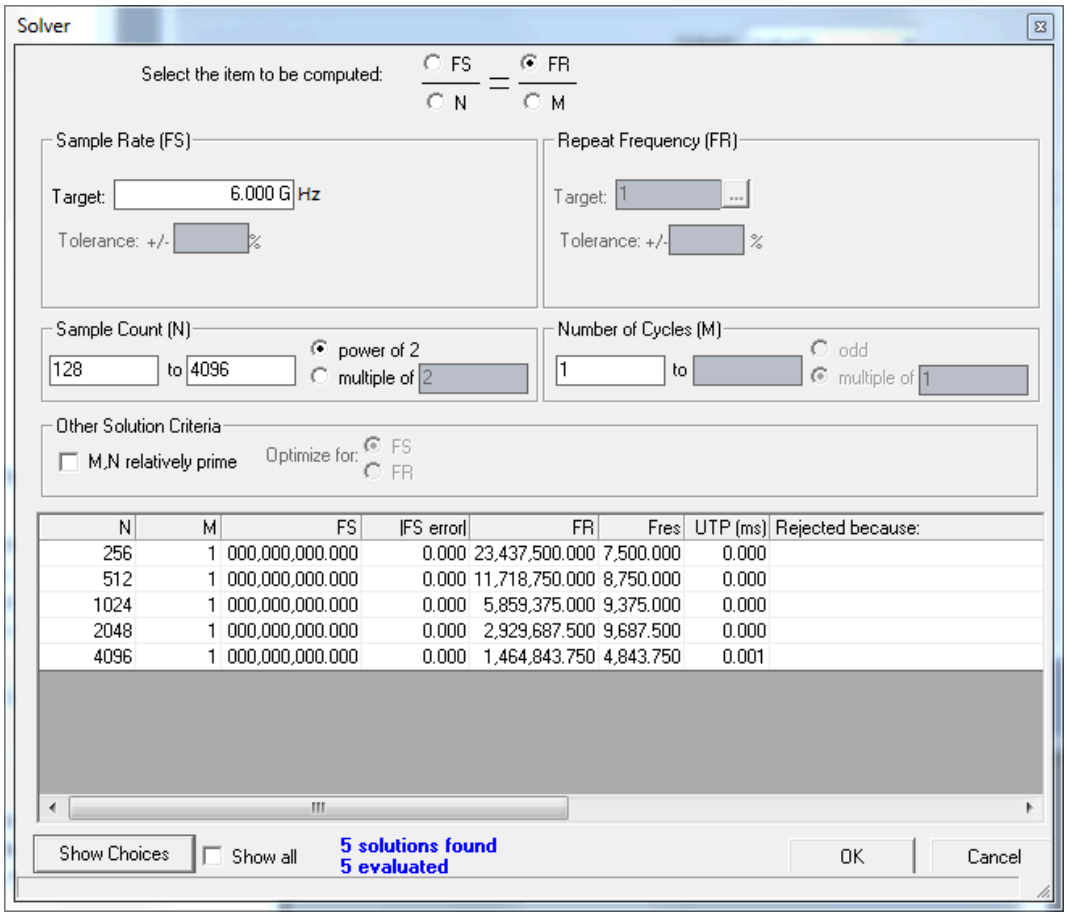
For instruments that can perform undersampling, determining timing solutions requires a different set of values and calculations. Therefore, the MSW displays a separate Solver window when you request timing solutions for an instrument that supports undersampling. This Solver also requests values as times instead of frequencies.

The following sections describe the MSW Solver for GigaDig:

- Parts of the MSW Solver for GigaDig Window
- Using the MSW Solver for GigaDig

Parts of the MSW Solver for GigaDig Window

The MSW Solver for GigaDig helps you create timing solutions based on a set of criteria and constraints you define. The Solver for GigaDig window includes the following controls.



Mixed Signal Workshop Solver for GigaDig

Characteristics of Signal to Capture

This section describes the waveform you want to capture and includes the following controls:

Repeat Period (Tr)	The amount of time it takes to source one iteration of a periodic waveform. Solver attempts to fill this field with a value from source instruments in the Mixed Signal Workshop. You can accept Solver's suggestion or fill in a value based on characteristics of the device under test.
Number of Cycles (M)	The number of iterations of the periodic signal you want to capture. This must be an integer, and must be at least 1.

Effective Sample Resolution

This section lets you set minimum and maximum sampling resolutions (Ts_{eff}). The first field specifies the smallest time between consecutive samples. The second field specifies the largest time between consecutive samples. Enter these values in units of seconds.

If you specify a value for N, these values are optional.

Number of Samples to Capture

This section specifies values that affect the number of samples to capture to represent the waveform accurately. It includes the following controls:

Sample Count (N)	This control includes two fields that specify, as a range, the approximate number of samples you want to capture per cycle.

	• The first field specifies the minimum number of samples you can capture to represent your signal accurately. This must be greater than twice the value of M. • The second field specifies the maximum number of samples. You can use this value and the value of M to limit the length of the capture. The total number of samples captured is N * M. The capture size must be smaller than or equal to the maximum number of samples the instrument can capture. (For the GigaDig, the capture size range is between 128 and 8388552 samples.) If you specify a range of values in Effective Sample Resolution, these values are optional.
Multiple	Specifies whether the number of samples must be a power of 2 or a multiple of a number you specify. This selection affects how you set up your DSP analysis. If you select multiple of, you must supply an integer value.

Optional Solution Criteria

This section specifies other criteria that affect the solutions and includes the following controls:

Allow Out-Of-Order Sampling	Specifies whether to allow or prohibit out-of-order undersampling in the Solver's solutions. Allowing out-of-order undersampling shortens capture time.
Maximum Capture Time (Sec)	The amount of time in which the capture must complete. Equivalent to Mixed Signal Workshop's UTP (Unit Test Period). This value reflects the time that the instrument spends capturing the signal and does not include time required to move or process the captured data. This value is optional.
Real Fs	Allows you to set lower and upper bounds for the actual sampling rate. These must be within the instrument's sampling range. For the GigaDig, this range is 10MHz to 20MHz. This value is optional.

Results

The results area lists the Solver's solutions based on your selections in the window. This includes the following controls:

Results list	Solver displays matching results in a table. Click any column heading in the Results Table to sort the entries according to that column. Click a solution to select it.
Show Choices	Tells Solver to produce a list of solutions that satisfy the values and constraints you specified.
Show All	Determines whether the table should contain all generated solutions, including those that did not meet one or more constraints, or only those solutions that satisfy all constraints you specified.
Results summary	Displays, in blue text, the number of solutions that matched your values and constraints and the total number of solutions that Solver considered.
OK	Applies the selected values for N, M, and Fs to the GigaDig capture block on the Mixed Signal Workshop. You must select Save to Excel in the MSW to save these values to the Mixed Signal Timing sheet.
Cancel	Dismisses the Solver without applying a solution to the Mixed Signal Workshop.

For the MSW Solver for GigaDig, the Results Table's FR error column represents the error that results from Microsoft Excel's precision limitations. (For the non-GigaDig Solver, this column represents the difference between the value required to make the solution perfectly coherent and the nearest frequency that the instrument can achieve.) Because the GigaDig's high precision clocking allows it to achieve precisely any frequency you can represent using a double-precision value, there can be no coherence error. However, Microsoft Excel limits the precision of values stored in its cells to 15 digits. The FR error column of the Results Table shows the error between the value represented as a double-precision value and the value represented as a 15-decimal-digit number. Solver stores the 15-digit value to the MSW.

Using the MSW Solver for GigaDig

The following is the general procedure for using the Solver to build an instrument's timing solution:

1.	Start the MSW Solver for GigaDig.

Click Solve in a GigaDig Capture box on the Mixed Signal Workshop.

2.	Enter a value for Repeat Period (Tr) or verify the value that the Solver suggests.
3.	Enter a value for Number of Cycles (M).
4.	Enter values for the range of effective sampling resolution. These values constrain the solutions that the Solver provides. Enter either, both, or neither.
5.	Enter values for Sample Count (N). These values constrain the solutions that the Solver provides. Enter either, both, or neither.
6.	Determine whether to allow or prohibit out-of-order undersampling.

For in-order undersampling and real-time sampling, clear the check box.

7.	Enter a value for Maximum Capture Time (Sec). This value constrains the solutions that the Solver provides and is optional.
8.	Enter values for Real Fs. This value constrains the solutions that the Solver provides and is optional.
9.	Set any other criteria.
10.	Click Show Choices.
11.	Select the most advantageous solution.
12.	Click OK.

While the Sample Count (N) and Effective Sampling Resolution (Ts_{eff}) ranges are optional, you must provide information for at least one of the ranges. You can specify any of the following combinations:

- Minimum and maximum values for Sample Count (N)
- Minimum and maximum values for Effective Sampling Resolution
- Minimum values for Sample Count (N) and Effective Sampling Resolution
- Maximum values for Sample Count (N) and Effective Sampling Resolution

Solver can calculate the omitted values using the following formulas:

- Minimum Ts_{eff} = (Tr * M) / (maximum N)
- Maximum Ts_{eff} = (Tr * M) / (minimum N)

If you use both Sample Count (N) and Effective Sampling Resolution ranges, the Solver uses the smallest range that satisfies both sets of constraints. That is, it uses the intersection between the Sample Count (N) and Effective Sampling Resolution ranges.
When you click OK, the Solver returns the following information and exits:

- Actual sample rate (Fs)
- Effective resolution (Ts_{eff})
- Effective sample rate (Fs_{eff})
- coherence error due to rounding (Ts_{err})
- Number of samples to capture (N)
- Total capture time (UTP)
- Undersampling ratio (USR), if you permitted out-of-order undersampling
- Required number of cycles of the periodic waveform (M)
- Frequency with which the periodic waveform repeats (F_r)

---

gigadig_prog.3.19

< >

GigaDig Programming : Determining Sampling Parameters : Determine Sampling Parameters Independently

Determine Sampling Parameters Independently

You can use Mixed Signal Workshop's Solver to determine the most efficient sampling parameters for the signal you want to capture. This is the preferred method of finding these values, as Solver ensures signal coherence and provides you with a range of solutions. In some cases, however, you may want to determine these parameters without using Solver.

This section includes the following topics:

- Determining Sample Size
- Determining the Sampling Rate

Determining Sample Size

The sample size is the number of samples you want to capture and store after a pattern start. This depends on the sampling rate, the required resolution, and the measurement type. For an amplitude measurement, select a sample size and a resolution for your application.

Determining the Sampling Rate

You must specify the sampling rate before a capture can occur. You can derive an effective sample rate from the programmed .SampleRate property and the input frequency. The GigaDig can perform undersampling of a signal, which allows you to capture samples at an effective rate faster than the actual rate at which the hardware can take samples.

Choose an effective sampling rate so that you capture:

- Enough samples per bit to describe the waveform accurately (the effective sampling rate is 1/resolution)
- An integer number of samples per block to define the entire waveform

How you program the .SampleRate property depends on whether you are performing in-order capture or out-of-order capture:

- Finding a Sample Rate for In-Order Capture
- Finding a Sample Rate for Out-of-Order Capture

Finding a Sample Rate for In-Order Capture

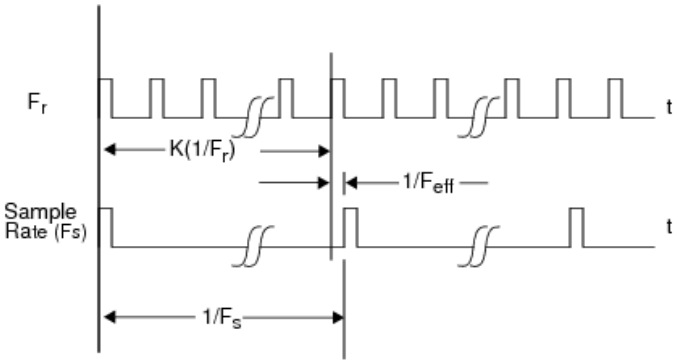
The equation for determining the sampling rate is:

$$\frac{1}{\text{sample rate}} = K\left(\frac{1}{F_r}\right) + \frac{1}{F_{eff}} \quad (2)$$

where:

F_r	Repetition frequency of the input waveform. For time domain calculations, Tr = $1/F_r$.
F_{eff}	Desired effective sampling frequency. For time domain calculations, $T_{eff} = 1/F_{eff}$.
sample rate (F_s)	Actual sample rate (for GigaDig, this must be between 10MHz and 20MHz). For time domain calculations, Ts = $1/F_s$.
K	Number of cycles of F_r required to achieve a sample rate between 10MHz and 20MHz.

The following figure shows these values in relation:



Effective Sample Rate

The following example illustrates determining the sample rate using in-order undersampling. This example allows you to choose a sample rate for a 128MHz repetitive waveform so that you capture 32 samples per cycle. The value of the actual sample rate must be between 10MHz to 20MHz. Based on the coherence equation:

$F_{eff} / N = F_r / M$

or:

$F_{eff} = F_r * N / M$

This works out to:

F_{eff} = (128MHz * 32) / 1

or:

F_{eff} = 4.096GHz

Since the sample rate of the GigaDig is between 10MHz and 20MHz, you must solve the above equation for the minimum and, if you wish, maximum values of K as follows.

The minimum value for K is:

$K_{(min)} >= F_r/sample\_rate_{(max)}$

$>= 128MHz/20MHz$

$>= 6.20$

The maximum value for K is:

$K_{(max)} <= F_r/sample\_rate_{(min)}$

$<= 128MHz/10MHz$

$<= 12.8MHz$

A noninteger value of K results in out of order samples in the capture. To create an in-order capture, round the value of K to an integer: round up for $K_{(min)}$ and round down for $K_{(max)}$. This produces new values of K as follows:

$K_{(min)} = 7$

$K_{(max)} = 12$

Using the minimum value for K in the above equation yields the maximum sample rate, and therefore requires the least capture time.

(3) $\frac{1}{\text{sample rate}} = \frac{7}{128\ \text{MHz}} + \frac{1}{4.096\ \text{GHz}} = \frac{1}{18.204\ \text{MHz}}$

Verify that your sample rate (F_s) is possible given the minimum resolution of the instrument. (For the minimum resolution, see the GigaDig specifications.) If it is not, round to the nearest possible frequency and recalculate to produce a final effective sample rate.

Finding a Sample Rate for Out-of-Order Capture

To find a sampling rate for out-of-order undersampling:

1. Collect or determine some information about the repeating pattern you want to sample.

For a digital pattern, you need the following:

- Total number of bits in the pattern
- Number of samples per bit
- Data rate of the pattern, in bits per second (bps)

For an analog pattern (waveform), you need the following:

- Pattern frequency
- Number of samples to capture over the entire pattern

2. For digital patterns, calculate the total number of samples in the pattern, N:

(samples per bit) * (total bits in pattern)

3. Calculate the time between effective samples, Ts_{eff}:

For digital patterns, Ts_{eff} = 1 / (pattern data rate * number of samples per bit).

For analog patterns, Ts_{eff} = (pattern frequency) / (total number of samples).

4. Calculate the minimum number of effective samples to skip between actual samples, β. β corresponds to the undersampling ratio (USR) modulo N. When N is larger than the USR, the USR is β. Since the USR = F_{eff} / Fs, figure β as follows:

- To produce the fastest available actual sampling rate, calculate a β_{min} that is at least 1 / (Fs_{max} * Ts_{eff}) between samples. For the GigaDig, Fs_{max} is 20MHz.

- To produce the slowest available actual sampling rate, calculate a β_{max} that is less than 1 / (Fs_{min} * Ts_{eff}) between samples. For the GigaDig, Fs_{min} is 10MHz.

5. Adjust β so that β and N are mutually prime (that is, their greatest common divisor is 1):

   - To find the fastest available actual sampling rate, increment β_{min}.
   - To find the slowest available actual sampling rate, decrement β_{max}.

Keeping β and N mutually prime ensures that the GigaDig captures each sample point exactly once.
β_{max} and β_{min} produce actual sampling rates that approach the limits of the GigaDig hardware. To use a different actual sampling rate, use any integer value for β between β_{max} and β_{min} provided that β and N are mutually prime.

6. Calculate the actual sampling frequency, Fs_{actual}:

$1/(\beta * Ts_{eff})$
For a function that performs these calculations, see Determine the Sample Rate for an Out-of-Order Capture.
The following example determines the sample rate using out-of-order undersampling and assumes that you want to find the fastest possible actual sample rate for a digital pattern.

1. You know the following information about your pattern:

   - The pattern contains 500 cycles, or 1000 bits
   - You want to capture 32 samples per bit
   - The pattern runs at 5Gbps, or 2.5GHz

2. The total number of samples to capture, N, is (samples per bit) * (bits in pattern), or 32 * 1000, or 32000.

3. The time between effective samples, Ts_{eff}, is (period of one bit) / (samples per bit).

   a. The period of one bit is 1/(bit rate). In this example, this is 1 / 5Gbps, or 2e-10.

   b. The time between effective samples, Ts_{eff}, is 2e-10 / 32, or 6.25e-12.

4. Calculate β_{min} so that it approaches 20MHz (β_{min} must be greater than 1/(20MHz * Ts_{eff})).

   a. β_{min} > 1/(20000000 * 6.25e-12)

   b. β_{min} > 1/1.25e-4

c. β_{min} > 8000

5. Since N (= 32000) and β_{min} (= 8000) have common divisors other than 1, start incrementing β_{min}. With β_{min} = 8001, N and β_{min} are mutually prime.

6. Fs_{actual} is 1 / (β_{min} * Ts_{eff}), or 1 / (8001 * 6.25e-12), or 19.997500MHz.

Verify that your actual sample rate (F_{actual}) is possible given the minimum resolution of the instrument. (For the minimum resolution, see GigaDig Specifications.) If it is not, round to the nearest possible frequency and recalculate to produce a final effective sample rate.
You can then program the GigaDig as follows:

1. Set .SampleRate to 19.997500MHz (Fs_{actual}).

2. Set .UndersamplingRatio to 8001 (β_{min}).

3. Set .ReorderedSamples to True.

You perform the same steps to determine the sample rate for an analog pattern (waveform):

1. You know the following information about your waveform:

   – The waveform frequency is 5MHz

   – You want to capture 32000 samples over the length of the waveform

Note that Tr is identical for analog and digital:
For the digital pattern, where the period of one bit is 200ps, Tr = 1000 bits * 200ps, or 200ns.
For the analog waveform, Tr = 1 / F_r, or 1 / 5MHz, or 200ns.

2. The total number of samples to capture, N, is 32000.

3. The time between effective samples, Ts_{eff}, is 1 / (pattern frequency * total number of samples). In this example, this is 1 / (5MHz / 32000), or 6.25e-12.

4. Once you figure the effective sampling rate, the calculations are the same for analog and digital patterns. Continue with step 4 above.

< >

file:///C:/Users/cpatle2/AppData/Local/Temp/chm_extract_ku0kmdve/combined.html 100/116

gigadig_prog.3.20

<	>

GigaDig Programming : GigaDig Pattern Control

GigaDig Pattern Control

You can control a GigaDig instrument using pattern microcodes, allowing changes to instrument states and parameters during a program. You can obtain repeatable timing by programming an instrument using pattern microcodes and parameter sets (PSets).

You create a pattern independently of the test program using the PatternTool or a text editor. Each digital vector in a pattern defines the data for each channel listed in the pattern, and can also define a pattern microcode that controls a separate instrument during pattern execution.

You can use PatternTool to help you identify such common pattern problems as:

- Too little space between microcodes
- Too little time between PSet calls
- Triggers occurring during a GigaDig capture

The following topics are available:

- GigaDig Pattern Microcodes
- GigaDig ASCII Pattern Syntax
- PSets

Related Topics

- GigaDig Pattern Microcodes
- GigaDig ASCII Pattern Syntax
- PSets
- Pattern Compiler

- Pattern Reverse Compiler

---

gigadig_prog.3.21

< >

GigaDig Programming : GigaDig Pattern Control : GigaDig Pattern Microcodes

GigaDig Pattern Microcodes

A microcode controls a single function, such as starting the capture of a signal. A series of microcode commands are available for use inside a pattern. Patterns allow you to precisely control the timing of a capture. Use patterns to establish repeatable timing or to synchronize with other instruments (such as the digital subsystem) in the tester.

The table lists the microcodes for issuing commands to a GigaDig instrument from a pattern.

GigaDig Pattern Microcodes

Microcode	Functional Description
Trig <SignalName>	The instrument immediately starts capturing the named capture signal. The Trig microcode takes as an optional argument the name of the signal to capture. If you omit the signal name, this triggers the default signal (see TheHdw.GigaDig.Pins.Signals.DefaultSignal)
Resync	Forces the capture channel clock into phase alignment with the Digital Subsystem T0 clock. See Synchronizing the Analog Clock.
PSet <PSetName>	Executes the specified parameter set. The PSet microcode requires the name of the PSet to apply. You must specify <PSetName>; there is no default PSet. See PSets.
Enable_Inst_Cond	Enables the instrument condition. The GigaDig asserts the instrument condition to the pattern generator when a capture completes. See Condition Bit Programming.
Disable_Inst_Cond	Disables the GigaDig from asserting the instrument condition.
Enable_Alarm	Enables all GigaDig alarms.
Disable_Alarm	Disables all GigaDig alarms.

☑ **Note:**

GigaDig microcodes have specific rules regarding the necessary spacing in vectors between microcodes on a specific channel. For more information, see GigaDig Specifications.

A parameter set (PSet) is a unique microcode that contains a set of values that describe a specific instrument setup. PSets allow a test program to change instrument setups within a pattern execution. Although a PSet is a hardware capability that changes the setup of an instrument during a pattern burst, you define it using VBT language and invoke it using PSet microcode.

Synchronizing the Analog Clock

The Resync microcode provides the ability to synchronize the tester's analog and digital clocks. It ensures that the first tick of all synchronized boards appears at a repeatable multiple of the digital subsystem clock. This allows you to create repeatable and portable timing solutions.

Synchronizing clocks does not affect the settings of any particular board with its own clock, including the GigaDig. Settings such as the sampling rate remain as you programmed them.

The Resync microcode affects the timing source for the entire GigaDig board, so this can change the timing on all channels that use the same clock. To ensure consistency in future applications, you should include a Resync microcode on the same vector for all channels that use the same clock. For example, if your pattern includes GigaDig channels cappos1, cappos3, and cappos5, you should include the Resync microcode for each of these channels.

See Microcode Restrictions for limitations on the Resync microcode.

Microcode Restrictions

The Resync microcode aligns the GigaDig clock with the digital subsystem clock to help ensure proper timing and repeatability. The UltraFLEX test system imposes the following restrictions on use of the Resync microcode:

- Use the same Resync arrangement in all patterns that you load at the same time. An "arrangement" is one or more instruments that use the Resync microcode. If all instruments use Resync, then you must resynchronize all instruments each time you resynchronize any of them. You can call the Resync arrangement more than once in a pattern, and the arrangement can exist in any, all, or none of the other patterns you load together.

The arrangement must be the same in all patterns you intend to load together. If you start a pattern that includes a different Resync arrangement, IG-XL returns a run-time error.

Before you load patterns that perform Resync independently, you must unload all patterns that synchronize together.

For example, you load Pat1, Pat2, and Pat3 together. Pat1 has Resync on pin cappos1 in vectors 1 and 5. Pat2 has no Resync. Pat3 has Resync on pin cappos1 in vector 3. This works because the Resync microcode appears on the same pins across all loaded patterns. But if you move the Resync to pin cappos2 in Pat3, the pattern fails.

- Do not place a Resync on the same vector as a Halt.

- Do not mix the PSet, Enable_Inst_Cond, Disable_Inst_Cond, and Resync microcodes across the channels in the same vector. These microcodes can exist on the same vector as a NOP. For example, do not put Enable_Inst_Cond on channel 1 and PSet on channel 2 in the same pattern vector. This applies for channels on the same GigaDig board hemisphere (channels 1 through 4 form one hemisphere, channels 5 through 8 form the second).

- Do not mix PSet names within a vector on channels on the same board. For example, if channel 1 calls PSet PS1, channel 3 cannot call PSet PS2 on the same vector.

You can mix Enable_Alarm, Disable_Alarm, and Trig microcodes on the channels of the same board in the same pattern vector. Trig labels can differ across channels also.

< >

gigadig_prog.3.22

< >

GigaDig Programming : GigaDig Pattern Control : GigaDig ASCII Pattern Syntax

GigaDig ASCII Pattern Syntax

GigaDig pattern microcodes are instructions you can place on an individual vector of a pattern. For that vector, a microcode instructs the GigaDig to perform a particular action, such as triggering a new capture.

Capture microcodes can be used only with pins that the instruments statement has specified as GigaDig. For example:

```
instruments = {
        DUT_AOUT:GigaDig;
}
```

For more information see Instruments Statement in the Pattern Language section.

Vectors that contain a GigaDig microcode other than nop execute from SRM.

The format for specifying a GigaDig microcode on a vector is:

```
(pin-list : GigaDig = microcode)
```

The following code shows the ASCII pattern microcode syntax for GigaDig instruments.

```
import_all_undefineds = yes;
import tset t1;
instruments = {
        GDIG_CH1P:GigaDig 1;
        GDIG_CH2P:GigaDig 1;
        GDIG_CH3P:GigaDig 1;
        GDIG_CH4P:GigaDig 1;
        GDIG_CH5P:GigaDig 1;
        GDIG_CH6P:GigaDig 1;
        GDIG_CH7P:GigaDig 1;
        GDIG_CH8P:GigaDig 1;
}
srm_vector
hsd2gdig_module ( $tset, (HSD_1, HSD_2, HSD_3, HSD_4, HSD_5, HSD_6, HSD_7, HSD_8))
{
start_label begin:
/* Synchronize GigaDig and HSD clocks for all channels */
 > t1     11111111 ;
((GDIG_CH1P, GDIG_CH2P, GDIG_CH3P, GDIG_CH4P, GDIG_CH5P, GDIG_CH6P, GDIG_CH7P, GDIG_CH8P) = Resync)
/* Allow enough time for all channels to synchronize */
 > t1     11111111 ;
repeat 16000
```

```
/* Enable the Instrument Condition Bit for all channels */
 > t1     11111111 ;
((GDIG_CH1P, GDIG_CH2P, GDIG_CH3P, GDIG_CH4P, GDIG_CH5P, GDIG_CH6P, GDIG_CH7P, GDIG_CH8P) =
Enable_Inst_Cond)
/* Trigger the capture specified by the signal "GigadigCapture" */
 > t1     11111111 ;
((GDIG_CH1P, GDIG_CH2P, GDIG_CH3P, GDIG_CH4P, GDIG_CH5P, GDIG_CH6P, GDIG_CH7P, GDIG_CH8P) = Trig
GigadigCapture)
 > t1     11111111 ;
 > t1     11111111 ;
loop1:
 > t1     11111111 ;
 > t1     11111111 ;
/* Loop until GigaDig sets the Instrument Condition Bit after capture */
branch_expr = (!inst_cond) > t1     11111111 ;
if (branch_expr) jump loop1 > t 1     11111111 ;
 > t1     11111111 ;
/* End the pattern */
halt > t1     11111111 ; }
```

< >

2015 Teradyne, Inc. All rights reserved.

gigadig_prog.3.23

<	>

GigaDig Programming : GigaDig Pattern Control : PSets

PSets

You can use parameter sets (PSets) to control the GigaDig from a pattern. PSets allow you to configure the GigaDig from within a running pattern, providing repeatable timing.

What is a PSet?

A PSet is a named set of property values that you program using VBT and implement in hardware using either a VBT statement or a pattern microcode. This allows you to define a set of hardware settings once and apply the settings repeatedly.

Each GigaDig PSet contains a value for each of the following parameters:

- SampleSize
- SampleRate
- InputDelay
- BlockCount
- BlockSpacing
- ExtraSamples
- ReorderedSamples
- UnderSamplingRatio
- VoltageRange

IG-XL uses default values for any property you do not explicitly program. For a description of the default values for GigaDig properties, see GigaDig Reset Values.

When you call a PSet from a pattern, you must always name the PSet you want to call; you cannot specify a default PSet.

GigaDig PSet Memory

Each GigaDig channel has enough PSet memory to store 157 unique PSets. Each GigaDig board hemisphere (channels 1 through 4, channels 5 through 8) has enough PSet memory to store 315 board PSets. If you apply 157 different PSets to the first GigaDig channel, the remaining three channels on the first half of the board are limited to 158 (that is, 315-157) PSets. You can conserve PSet memory by using a single GigaDig board to capture data from multiple sites (for example, connect channel 1 to Site 0, connect channel 2 to Site 1, and so on), then using one PSet to assign the same values to all parameters across all sites.

PSet Error Checking

Because you define PSets dynamically during a test program run, validation does not include error checking or processing. This happens when you start the pattern that contains a PSet. At that time, IG-XL verifies that you have defined all signal and PSet names that appear in the pattern. If you leave any PSets or signals undefined, IG-XL returns a run-time error that notes the name of the pattern that contains the undefined item and the name of the procedure or the VBT function from which you call the pattern.

Do not configure PSets during a pattern burst, as this can lead to a conflict between the Visual Basic programming and implementing the PSet in hardware. If this happens, IG-XL returns a run-time error.

PSet Programming

To create a PSet and assign values to the PSet's properties, use the PSets VBT property. For more information, see TheHdw.GigaDig.Pins.PSets in the VBT Syntax Reference.

To program hardware with the values in a PSet, use:

- The PSet microcode in an HSD pattern (see GigaDig Pattern Microcodes)
- The VBT method TheHdw.GigaDig.Pins.PSets.Item.Apply

For an example of GigaDig programming using PSets, see Capture a Signal Using Pattern Control and PSets.

< >

2015 Teradyne, Inc. All rights reserved.

gigadig_prog.3.24

< >

GigaDig Programming : Using GigaDig in Concurrent Testing

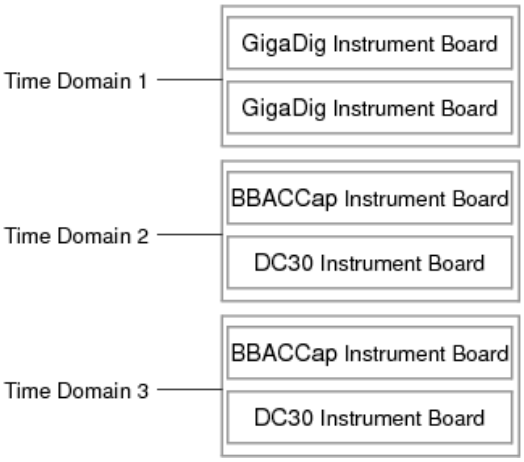
Using GigaDig in Concurrent Testing

To support concurrent testing, you can assign GigaDig channels to a time domain. This means that GigaDig instruments can be part of a concurrent pattern burst. However, only one pattern in the concurrent burst can contain GigaDig instruments in it at one time. This limitation means that GigaDig channels cannot be in more that one time domain for each concurrent burst.

Concurrent features for GigaDig include:

- A GigaDig board can be configured for one time domain
- Multiple GigaDig instruments can be grouped into a common time domain
- GigaDig is restricted to using the Global flow domain and synchronous pattern starts

The following figure shows a configuration with two GigaDig instruments assigned to its one time domain and other UltraFLEX instruments assigned to other time domains.



GigaDig Concurrent Testing Configuration Example

For more information, see About Concurrent Test.

< >

gigadig_theory.2.1

< >

GigaDig Theory of Operation

GigaDig Theory of Operation

The GigaDig is an undersampling digitizer with eight differential inputs and PPMU capability. The GigaDig instrument comprises the following subassemblies:

- Main board
- Two rider boards with four differential channels each
- Power board
- Sidewalk 2 delivery path interface

The following topics are available:

- GigaDig Board Components
- DSP Service
- GigaDig PPMU

For information on GigaDig instrument performance specifications and standards, see GigaDig Specifications.

< >

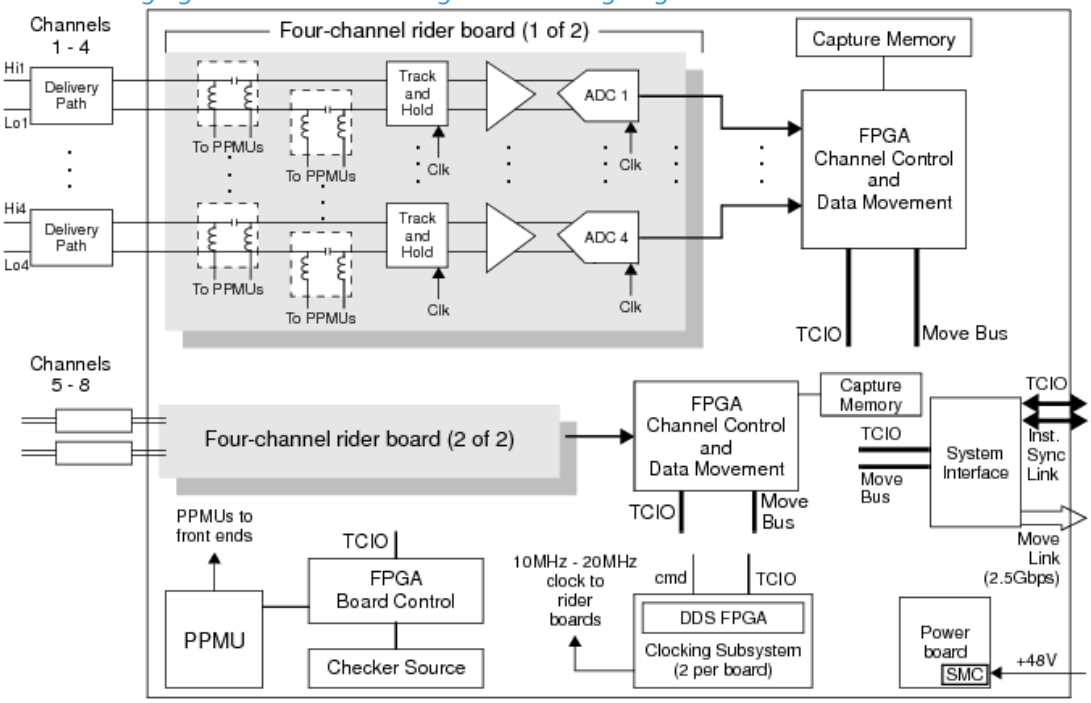
gigadig_theory.2.2

< >

GigaDig Theory of Operation : GigaDig Board Components

GigaDig Board Components

The following figure shows a block diagram of the GigaDig.



GigaDig Block Diagram

The GigaDig receives signals through a high performance Pogo interface on the Sidewalk 2 portion of the DIB. Cabling delivers the signals from the Sidewalk 2 interface to the input of the rider board. The rider board contains the input stage, track and hold, ADC, and clock distribution. The sample clock is common to all channels but the input stages to the ADCs are independent for each channel. The input signal is AC-coupled and connects directly and permanently to the DIB; there are no disconnect relays for the analog input stage. In the input stage, each GigaDig channel includes a bias-tee that allows for PPMU connections to the DIB.

The sample clock, ADC, and PPMU signals from each of the two rider boards connect to the main board. The main board hosts the following functions:

- Clock generation

The clock generation uses direct digital synthesis (DDS) to create a 10MHz through 20MHz sample clock from the 100MHz system reference clock, then distributes the sample clock to the two rider boards.

The clock controller also:

- Manages synchronization information from the Instrument Sync-Link (ISL) signal
- Passes microcodes to each channel

- Capture memory

During a capture, the GigaDig stores the data from each of the eight inputs in a separate capture memory (CMEM) bank. Once the capture completes, the GigaDig automatically moves captured data either to the DSP computer, if one is present, or to the host computer if no DSP computer is available. Transfers to the DSP computer take place over the MoveLink; transfers to the host computer take place over TCIO.
If you captured data out of order, GigaDig software can reconstruct the data into a consecutive data set. To do this, you must assign a value to the .UndersamplingRatio property and set the .ReorderedSamples property to True.

- Checker source

An on-board checker source with a Direct Digital Synthesizer allows the GigaDig to perform an internal functional check of the board when called from checkers.

- Test system interface

The system interface controller manages connections to the high-speed links for moving data to the DSP Processor (MoveLink) and synchronization (ISL). The interface also uses the TCIO data bus for control and instrument status communication.
A separate power board converts the UltraFLEX's 48V DC input to the voltages required for the main board and each of the rider boards. The power board contains the System Monitor Controller (SMC) that controls GigaDig power up and sequence and monitors the power board supplies when the instrument is on.

- PPMU

The main board contains the PPMU circuitry to control sixteen PMU channels (two per differential input). The capability of the GigaDig PPMU is similar to that of the High Speed Digital PPMU.

< >

gigadig_theory.2.3

< >

GigaDig Theory of Operation : DSP Service

DSP Service

This section describes using the GigaDig to capture and prepare data for DSP.

The GigaDig supports DSP analysis of captured data using either the DSP computer or the host computer. You can use the DSP Run Mode option to select which computer analyzes your captured data. This allows you to debug DSP procedures on the host computer.

On software power up reset, IG-XL determines whether a DSP computer exists. If it does, the GigaDig automatically uses instrument-initiated move (IIM) to move data to the DSP computer after a capture. If the DSP computer does not exist, IG-XL disables IIM for all GigaDig boards. After a capture, the data remains in capture memory (CMEM) until your DSP procedure requests it. At that time, IG-XL transfers data from CMEM to the host computer using TCIO. You need not program the instrument differently, and there is no difference in behavior, except that TCIO moves data more slowly than IIM does.

After retrieving your data, IG-XL performs DSP preprocessing, then calls your DSP procedure. The GigaDig does the following DSP preprocessing:

- Scaling: IG-XL scales the captured samples by the voltage range setting and any applicable calibration factor.
- Reordering: If you perform out-of-order undersampling, DSP preprocessing reorders the samples according to the value of the UndersamplingRatio property. (You must also set the ReorderedSamples property to True.) The DSPWave contains the samples in the correct order.
- Setting SampleRate: If you have set the UsingEffectiveSampleRateInDSPWave property to True, the GigaDig DSP property is set to the effective sample rate.

For more information on performing DSP using IG-XL for UltraFLEX, see About Digital Signal Processing (DSP).

< >

gigadig_theory.2.4

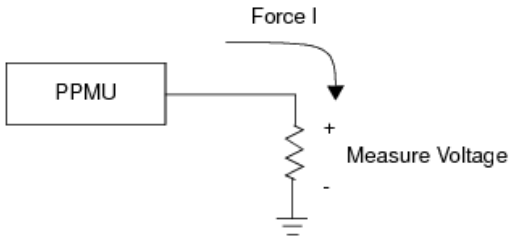
< >

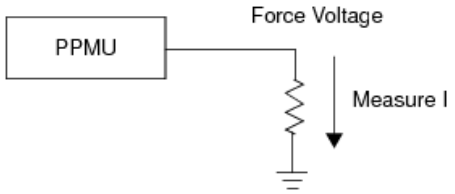
GigaDig Theory of Operation : GigaDig PPMU

GigaDig PPMU

Every GigaDig input is connected to a four quadrant force-voltage and measure-current or force-current and measure-voltage pin parametric measurement circuit (PPMU). The PPMU can be used to:

- Force voltage while measuring current (for leakage measurements)
- Force current while measuring voltage (for open/shorts tests)





PPMU Usage

The capability of the GigaDig PPMU is similar to that of the High Speed Digital PPMU. For more information about the PPMU, see About the PPMU.

< >